

ANR SCALER



2023-2026

D 3.1

State of the Art in Microservice ADL and Microservice Auto-Scaling Strategies

Chef de file pour le D 3.1 :
Co-rédacteurs :

Philippe MERLE
Hugo MONFLEUR,
Vania MARANGOZOVA,
Wassim AROUI

February 29, 2024



Contents

1	Introduction	3
2	Microservice Architecture Description Languages	8
2.1	ADLs for Microservices Applications	8
2.1.1	ADL Concepts and MSA	8
2.1.2	Concerns	9
2.1.3	Design Patterns	12
2.2	Taxonomy	16
2.2.1	Classification Criteria	16
2.2.2	Identified Solutions	19
2.2.3	ADL Classification	29
2.2.4	ADL Tools Classification	38
2.3	Research Challenges	44
3	Microservice Auto-Scaling Strategies	46
3.1	Background	46
3.1.1	Autonomic Computing, Self-optimisation, and Auto-Scaling	46
3.1.2	Elasticity in Cloud Computing Taxonomy	46
3.2	Microservice Auto-Scaling Strategies	47
3.2.1	Background	47
3.2.2	State of the Art on Microservice Auto-Scaling	48
3.2.3	Research Challenges	53
3.3	Cloud-Native Network Function Auto-Scaling Strategies	54
3.3.1	Background	54
3.3.2	State of the Art on Cloud-Native Network Function and Microservice Auto-Scaling	54
3.3.3	Research Challenges	56
4	Concluding Remarks	57
A	Classification of ML-Oriented Works [51]	58



List of Figures

1.1	MSA taxonomy [7]	4
1.2	SOA taxonomy [7]	5
2.3	MSA classical metamodel [24]	11
3.4	MAPE-K Architecture from [44]	46
3.5	Elasticity Taxonomy from [42]	47
3.6	Auto-scaling Classification Taxonomy from [52]	48
3.7	Auto-scaling Classification Taxonomy from [51]	50
3.8	ML Orchestration Techniques from [51]	51
A.9	Works on Workload Characterization of Microservices from [51].	58
A.10	Works on Performance Analysis of Microservices from [51].	59
A.11	Works on Anomaly Detection of Microservices from [51].	60
A.12	Works on Dependency Analysis of Microservices from [51].	61
A.13	Works on Dependency Analysis of Microservices from [51].	62

List of Tables

1	Principal goals and targets of the existing solutions	29
2	Comparison of components (microservices) description in Microservices ADLs.	30
3	Presence of connectors and architectural configurations in microservice ADLs.	33
4	Presence of other microservices systems aspects in microservice ADLs	35
5	Main decomposition and focus of microservice ADLs	38
6	Presence of implementation tools for identified solutions	39
7	Existing architectural analysis tools	41
8	Existing visualization tools	42
9	Other ADL tools	43



1 Introduction

Microservice adoption has been steadily growing in recent years. A 2020 O'Reilly survey on more than 1500 software engineers indicated that 77% of them were using microservices, 61% for more than a year, and 29% of them worked for companies that were in the process of migrating or implementing the majority of their systems using microservices [1]. Another, more recent, report from O'Reilly indicates that while the growth in popularity of microservices as a topic tends to slow down, it is the second most popular topic in software architecture [2].

Monolithic Applications Microservices have emerged from the industry in the early 2010s to respond to what was until then the dominating application style i.e. *monolithic* applications. The latter are to be considered as units of execution where all the application logic is being expressed in a single chosen language through its specific concepts with strong coupling. As Dragoni puts it in [3] "A monolith is a software application whose modules cannot be executed independently." Because of that the support for the execution of a monolithic application is necessarily a single machine (physical or virtual) and horizontal scaling is done by replicating the whole application[4].

The nature of monolithic applications is the source of multiple technical issues among which we can distinguish those that are relative to development and those that are relative to deployment and operation.

On the development front we note that monoliths grow to become large programs [5] making them "difficult to maintain and evolve" [3]. Multiple modules involve multiple libraries and because of that "adding or updating libraries results in inconsistent systems" [3] or imply important and costly code updates[5]. Finally having all parts of the code linked together implies a "technology lock-in for developers, which are bound to use the same language and frameworks of the original application" [3, 5].

On the deployment and operation side, the necessity of a unique deployment environment for the entire application forces a "one-size-fits-all configuration". This implies having to arbitrate about a global environment in which different modules with different requirements will live. As the application has at its root a single process, any modification or failure implies a complete reboot provoking considerable downtime.

The implications of these limitations are not only technical but reflect on the developing team organization. As it is not possible to dissociate the execution of the different parts of the code, any modification has to be validated by a large proportion of the team or at least by some central authority. Indeed any default could potentially render the entire system unusable. Because of that decisions are heavily centralized and necessitate the consultation of a large number of people to ensure that the changes won't have a problematic effect on their area [3, 5].

Service-Oriented Architectures (SOA) To tackle the limitations of monolithic applications, a new paradigm has emerged in the early 2000s called Service-Oriented Architecture (SOA). Put simply the idea is to decompose the "classical" monolithic application in multiple independent services modeled on real-world business activities. The inner workings of one service are unknown to one another and the interaction is done through exposed interfaces that services agree upon through "contracts" that define the operations provided and the means of communication to use them.

The core novelty of SOA is to propose a new design pattern that separates business concerns. The main SOA principles include [6]:

- Standardized Service Contract: document indicating how to interact with a service.
- Service Loose Coupling: Aiming to have on one front minimal dependency between clients



and the service contract and on the other front minimal - ideally none - dependency between the service contract and the concrete implementation.

- Service Abstraction: Expose as little as possible of service to its environment to allow for loose coupling.
- Service Reusability: Design services capabilities while being agnostic to functional contexts allowing for multi-purpose logic.
- Service Autonomy: Designing services having in mind "isolation levels and service normalization" to allow for availability, reusability, and scalability.
- Service Statelessness: Aiming at minimal statefulness to increase availability and scalability.
- Service Discoverability: Aiming for service accessibility and usability to be minimally dependent on the environment to enable maximum reusability.
- Service Composability: Design services so that they can be reused as parts of future compositions.

Microservice Architectures (MSA) and SOA The microservice architecture style (MSA) can be considered as a particular style of SOA. However, in its 2015 book [7] Richards notes multiple distinctions between SOA and MSA.

In terms of *service characteristics* the differences relate in particular to:

- Service Taxonomy. MSA distinguishes only between functional services and infrastructural services. Functional services provide application-specific business functionalities while infrastructural services provide technical functionalities that are nonspecific to a particular application.

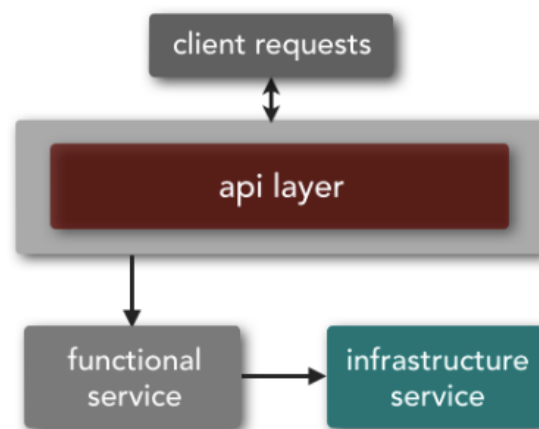


Figure 1.1: MSA taxonomy [7]

SOA has a much more developed taxonomy with four different types that can be ranked in levels of decreasing abstraction and increasing specificity. Business services are abstract services, void of implementation and define business operations at enterprise level. Enterprise services are concrete services implementing business services, they are usually "shared across the organization". Application services provide specific operations to enterprise services that are not business operations but are used to resolve them. Lastly, Infrastructure services provide non-business functionalities used to run the system.

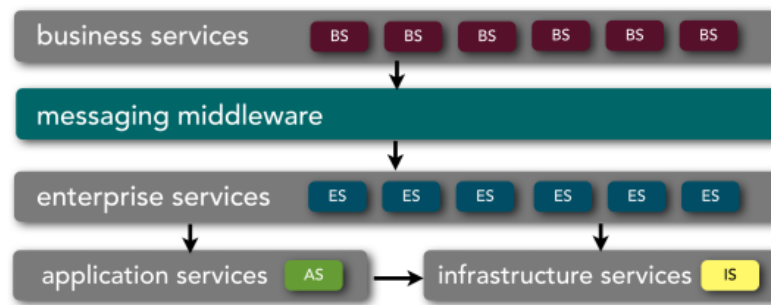


Figure 1.2: SOA taxonomy [7]

- **Service Ownership.** In SOA service ownership matches service taxonomy with separate owners for separate types. An important similarity with Monolithic applications is the distinction between business service teams and infrastructure service teams. The negative consequences of that separation are the same as in Monolithic applications development i.e. each decision implies the coordination of an important part of the organization.
- **Service Granularity.** SOA does not give particular guidelines relative to service granularity while MSA aims for one distinct capability per microservice. In practice, SOA services are akin to monolithic applications going "from small application to very large enterprise services." Regarding Microservices it is to be noted that the service granularity problem is then transmitted to the capabilities granularity problem.

There are the following distinctions between SOA and MSA architectures:

- **Component Sharing:** SOA tends to favor a "share-as-much-as-possible" approach grouping resources in shared services used by as many other services as possible and thus clarifying subtle differences in shared concepts by removing functionality duplication but also increasing coupling by centralizing service dependencies. MSA takes the opposite approach namely "share-as-little-as-possible" favoring repetition over coupling.
- **Orchestration vs Choreography:** SOA tends to favor orchestration of services i.e. having a central component controlling the general workflow while MSA is better suited for choreography, a style where the workflow is not centralized. In SOA, by design, a Middleware component (such as an Enterprise Service Bus) manages communication with the different enterprise services under which choreography can happen. The component responsibility can vary but it often manages message transformation so that a service can interact with another without knowing the precise input it needs. That allows for multiple communication protocols in an SOA application. Such a component is not fundamental in MSA where services must know how and where to request needed resources. Because of that the communication protocol must be unified throughout the entire MSA application and each microservice must know the exact contract of the other microservices it interacts with. It is to be noted that recent developments in microservices have brought orchestrating components e.g. Kubernetes.

MSA Gains and Pains Microservices are meant to solve or at least mitigate the difficulties encountered by monolithic and service-oriented architectures. The gains that style has brought can be classified into different categories[8]:

- **Design time:** The usage of "bounded contexts"[9][10] allows separation and encapsulation of the concepts of an application in smaller entities. This independence, with only some points of contact needing mutual understanding, brings easy governance, flexibility as well as it



allows the application to be fault tolerance by design and permits it to be cloud native.[8] The particular nature of microservice applications has led to the development of specific design patterns[11] that either bring positive gains or at least mitigate the particular difficulties linked to microservice architecture[8].

- Development: The department in which microservice architecture shines the most relatively to its alternatives. The focus on loose coupling permits the independence of teams in development and in choosing the technology stack. Also, the encapsulation of simple logic in microservices allows to easily integrate new members to the projects.[8][12][3]
- Operation: Loose coupling of microservices allows their independent deployment whether initial or as the application is online. Their deployment is made easy by the widely used container technology and its support by providers. Once online, they are fault tolerant in the sense that they can continue to run even if neighboring microservices are not which implies that their reboot is independent from the rest of the architecture.[8][12][3]

The biggest operation gain is the ability to scale microservices independently thus allowing for fine-grained scaling while the duplication of large components in the case of service-oriented architecture and of the whole application with monolithic architecture is required. Because of that microservices applications can provide better reaction time due to reduced deployment time and resource-saving due to the limitation of replicated microservices[8][5][3]. It is to be noted that those gains are relative to the ability to identify the microservices in need of scaling in a given situation.

While microservices are a promising idea that can solve or alleviate some difficulties encountered in monolithic or service-oriented architectures, they have also some considerable drawbacks. Some of them are fundamentally linked to the nature of microservices, mostly due to the difficulty of using microservices appropriately. The same classification used for the "gains of microservices" can be applied to the "pains of microservices"[8]:

- Design Time: While the technical foundation of microservices allows for the independence of microservices by design it also implies a global view of the application so that the independence given to teams does not mean harmful disorganization. The original idea of having one capability per microservice is tempting but often unclear or inefficient. Because of that finding the appropriate granularity for microservices is a difficult and risky task at design time[8][13][14].

The complexity of the architecture can also grow out of hand either by poor original segmentation design (such as having the above-mentioned inappropriate granularity) or because of unchecked growth in the creation of microservices. Managing that aspect, particularly when having to consider other difficulties inherent to microservices architecture such as security considerations or latency risks, can be challenging[8][13].

Security is a particular concern that requires more attention than it does with monolithic or service-oriented architectures since the fact that every microservice exposes an API increases the surface of attack upon which the mandatory access control is rendered more difficult due to the heavily distributed nature of microservice applications[8].

Lastly while modifying the architecture, the independence of microservices implies the necessity of backward compatibility for other microservices to be able to realize their operations thus creating a risk of endpoint proliferations[8][13].



- **Development:** While Development is the department where microservices shine the most, two aspects are made more difficult by the distributed nature of microservices: data storage (consistency, distributed transaction, heterogeneity, query complexity) and testing (performance and user experience)[8].
- **Operation:** Resource usage is a major concern in microservice architecture. Since intra-application communications go through it, network usage is where microservices have the biggest impact compared to other architectural styles.[8][13]. A lesser concern is the usage of compute resources that can be intensified due to the multiplication of runtime environments[8]. While it is generally possible to avoid single points of failure, cascading failures can still happen, service coordination and location are challenging causing difficulties in problem location that are increased by the scattering of logs in the application[8].

It is to be noted that most of the difficulties contained in microservice architecture come from their distributed nature and independence from each other. These difficulties can be alleviated by careful design and work on microservice architectures, which can leverage microservice-oriented ADLs.

Outline. This report is organized in two main parts. The first part, presented in Section 2 presents the state of the art on the languages for describing microservice architectures. To provide a meaningful classification, this section starts with a more detailed presentation of microservices' concepts. The second part of the report tackles the question of microservice scaling.



2 Microservice Architecture Description Languages

The name *Architecture Description Language* (ADL) appears to be self-explanatory. According to [15], *an ADL for software application focuses on the high-level structure of the overall application rather than the implementation details..* The ISO/IEC/IEEE 42010:2022 [16] defines ADLs as *means of expression, with syntax and semantics, consisting of a set of representations, conventions, and associated rules intended to be used to describe an architecture.* However, citing [17], *there's little consensus in the research community on what an ADL is.*

Instead of defining ADLs with an intuitive definition, it is common to consider their fundamental constituents. In short, an ADL describes the *components* of a system, the *connectors* between those components, and the general *properties* of the system. It must define explicitly the points of contact between the constituents of the system and allow for system evolution. It is also required to be able to express the system rules and ensure that they are respected.

ADL Tools. ADLs usually come with associated tools that simplify their usage [17]. These tools are typically used for code execution, system analysis and system representation.

- **Code execution.** ADLs are either executable themselves or provide tools to produce executable code[18][19][17]. Doing so solves the problem of having to maintain the architectural description alongside the executable code. For a language to be an ADL and thus provide a sufficient level of abstraction, it has to allow for different target implementations[19].
- **System Analysis.** ADLs may come with tools that can be offline (static analysis) and apply to the architecture description, or online (dynamic analysis) and check the properties of the system as it's running. Offline tools verify properties, such as those expressed in the components, connectors, and configurations descriptions but may also simply check whether the system's description is coherent[19]. They can also help with code generation [17]. Online analysis can be used to check if properties are respected at runtime or to extract the architecture from the running system.
- **System Representation.** The last family of ADL tools is used to represent the architecture. It generally consists in an abstraction of the system with an adaptable level of detail. The representation tools produce different views of the system to fit the informational needs of different situations and stakeholders[17][19].

2.1 ADLs for Microservices Applications

2.1.1 ADL Concepts and MSA

Considering that "a microservice architecture is a distributed application where all its modules are microservices"[3], mapping ADLs concepts to microservice applications is straightforward enough.

The components are the microservices. The descriptions of the interfaces are usually a list of their endpoints but can also be seen at a higher level showing exposed ports. Typing is a possibility, particularly for infrastructural microservices that are common in multiple architectures. Default types are harder to think about for functional microservices but can be defined by users for a particular system or family of systems. Implementation instructions can be added by specifying a particular technology for a general microservice type. Constraints can be expressed at microservice level by indicating such requirement as the type of protocol used for a connection (e.g. REST), connections with specific types of microservices (e.g. a database, a registry, a message broker) or being part of a larger structure (e.g. a registry being inserted in the "right" spot of a registry pattern).

The connectors are by essence more difficult to map to an object as they are an abstraction of the



communication process happening between multiple entities. In a microservice context, besides the connected microservices, a key point is the protocol used to communicate. A type can also be relevant for connectors as simple as "one to one" or "one to many". Some abstract structures such as design patterns could then require some particular type of connectors. For instance a registry pattern would require a connector of the "one to many" form between the registry microservice and the others while a dedicated database pattern could require a "one to one" type of connector. Constraints on a connector could be a technical requirement such as a minimal bandwidth or a particular type of microservice to be connected to other. That last example somewhat blurs the distinction between connectors and architectural configurations as through semantics and constraint expression a connector can contain the logic of a particular pattern.

The configuration in a microservice context are identified groups of microservices linked with connectors in such a way that they represent a functional entity. They can be seen as the formal representation of design patterns and the constraints they induce. As such they can be of different natures. An instance can be the topology of a particular organisation (e.g. registry pattern, gateway pattern). Another architectural configuration describing the same object from a different point of view can address the workflow of such a topology, indicating the path of communication. A more specific architectural configurations can represent the workflow of a microservice system at a particular point in its life cycle (e.g. deployment). Architectural configurations are the elements of an ADL that bring the most in terms of understandability, compositionality as they reveal the hidden logic of an application. Because of that they are also they principal structure destined for analysing (online or offline) and representation.

2.1.2 Concerns

Definition While it is widely used in software development, the concept of "concern" doesn't have a clear definition. Tarr and Osser give a fitting description of that situation: "'Concern' is a concept of which most software engineers have an intuitive grasp, but no precise definition. This is largely because there is such a variety of concerns of interest." [20]. A concern is then something defined through its "relevance to a particular stakeholder" [16] and then, identifying that concern and which parts of a system are relevant is done, as Dijkstra puts it, by "focussing one's attention upon some aspect". Once the aspect is identified the concern is "a predicate on units that indicates whether or not a given unit pertains to that concern" [20].

But if the concept of concern seems too large so that its particular instance could be encapsulated in a definition, [16] give an interested perspective on the domain it covers as it considers ultimately a concern as a matter of relevance relatively to the "subject of an architecture description". Then a concern is essentially a particular view on an architecture.

To give an idea of the broadness of what can fall under that definition in a microservice context, we can first think of common matter such as logging, security, scalability, maintainability, monitoring or design patterns. But then any of these large domain of interest can contain more precise concern. For instance, any design pattern such as "registry pattern" or "single database per microservice" is in itself a concern, a point of view on the system where the relevant elements are the microservices and their connections participating to this particular design pattern.

Another concern of a different nature from the other listed before could be particular naming conventions where the elements relevant to them are particular properties of microservices or connectors on which the naming convention is applied.

Those are aspects of the architecture in the sense that they express "the fundamental concepts or properties" as well as "the governing principles for the realization and evolution" [16] of the system.



Crosscutting Concerns A cross-cutting concern is a concern that affects several components of a system and intersects with other concerns. The first phenomenon is called *scattering*, while the second is *tangling*[21].

- *Scattering*. If a given concern is found in multiple components of the system, is said to be scattered. The metric for quantifying the scattering of a concern or a system relies on proportionality between the number of components affected by a concern and the total number of components in a system[22].
- *Tangling*. If there are multiple concerns in a single component of the system, the component is said to be tangled. The metric for quantifying the tangling of a component or a system relies on proportionality between the number of concerns affecting the component and the total number of concerns[22].

Depending on the language used in the system, the resulting system usually puts forward a dominant concern and relegating others concerns to sub-descriptions which results in scattering and tangling. This is called the "tyranny of the dominant decomposition" [21]. For instance, the description of a program through a language whose concepts allow for data modularization would scatter code elements such as functions. Reciprocally in a procedural programming language, references to data structures are scattered among and tangled in procedural structures.

Concerns in Microservices Applications Microservices applications have specific crosscutting concerns. Some of them are largely immaterial such as team ownership of microservices and are not identifiable in code and description files. Others are scattered among the microservice description and require analysis to be exposed. The following Docker Compose code used to deploy the TeaStore benchmark application[23] is a good example of that phenomenon. The colors are used to distinguish concerns.

```
1 version: '3'
2 services:
3   registry:
4     image: descartesresearch/teastore-registry
5     expose:
6       - "8080"
7   db:
8     image: descartesresearch/teastore-db
9     expose:
10      - "3306"
11     ports:
12      - "3306:3306"
13   persistence:
14     image: descartesresearch/teastore-persistence
15     expose:
16      - "8080"
17     environment:
18       HOST_NAME: "persistence"
19       REGISTRY_HOST: "registry"
20       DB_HOST: "db"
21       DB_PORT: "3306"
22   auth:
23     image: descartesresearch/teastore-auth
24     expose:
25      - "8080"
26     environment:
27       HOST_NAME: "auth"
28       REGISTRY_HOST: "registry"
29   image:
30     image: descartesresearch/teastore-image
```



```
31 expose:
32   - "8080"
33 environment:
34   HOST_NAME: "image"
35   REGISTRY_HOST: "registry"
36 recommender:
37   image: descartesresearch/teastore-recommender
38 expose:
39   - "8080"
40 environment:
41   HOST_NAME: "recommender"
42   REGISTRY_HOST: "registry"
43 webui:
44   image: descartesresearch/teastore-webui
45 expose:
46   - "8080"
47 environment:
48   HOST_NAME: "webui"
49   REGISTRY_HOST: "registry"
50 ports:
51   - "8080:8080"
```

Listing 1: Crosscutting Concerns in the TeaStore Docker Compose file

There is an obvious scattering of the cross cutting concerns for this file. It is described in detail in [24].

The reason for the high level of scattering in the description of microservices' applications is a consequence of the widely shared microservice application base metamodel where "each Application owns a non-empty set of microservices. Each Microservice has a unique distinct identity msid (e.g. registry) and owns a non-empty set of properties. Each Property has a unique distinct name (e.g. image) and a value (e.g. descartesresearch/teastore-registry)"[24]. The system's unit of decomposition being the microservices, each cross-cutting concern is scattered among the microservices description blocks. In mirror of that, microservices' description blocks are subjects of a high level of tangling.

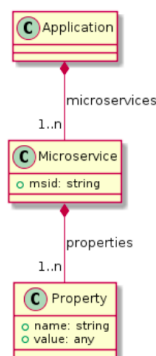


Figure 2.3: MSA classical metamodel [24]

The TeaStore example shows a variety of cross-cutting concerns that are identifiable from the analysis of the properties of the system. Others could be and are probably existent in the system but are not detectable in the system description. Some cross-cutting concerns could be specific to a particular application while others are frequently found in applications. Examples would be logging,



monitoring, security, fault-tolerance, naming conventions or design patterns. As *design patterns* define proven architectural solutions, they appear as a good starting point for concern identification.

2.1.3 Design Patterns

The microservice architecture style impacts numerous aspects including team organization, infrastructure management, code design and security concerns. Because of that a number of MSA-related design patterns have emerged. In the following we give a quick overview of some of the most common extracted from Richardson's book [11].

Application patterns

Decomposition Ways of decomposing an application into microservices.

- Decompose by Business Capability: Model the microservice application on the business activity of the company without spreading capabilities among multiple microservices but instead allocating one or several capabilities to strictly one microservice.
- Decompose by subdomain: Model the microservice application on the different subdomains that are key to it functioning. Some subdomains may not be mappable to business capability but necessary to perform business actions (e.g. specialized mathematical operations, data storage...). In such a decomposition such sets of related functionalities are grouped and implemented as a service.
- Self-contained service: Service does not wait on neighbor's answers for continuing its execution, queries are logged into a database so that even though a microservice can be unavailable it will be able to recover missed queries from it.
- Service per team: Each microservice is assigned to a unique team of a relatively small size.

Testing

- Service Integration Contract Test: Tests are written by the teams using the microservices, not the team developing it.
- Service Component Test: Having test doubles (analogous to mock class) for every microservice invoked as a result of test a particular microservice.

UI

- Server-Side fragment composition: Composition of the displayed webpage is done Server-Side.
- Client-Side Fragment Composition: Composition of the displayed webpage is done Client-Side.

Observability Audit Logging: Record user activity in databases.

Data Patterns

- Database Architecture
 - Shared Database: different microservices sharing databases. Usual ACID transactions are used making the system simpler to design, develop and operate. On the other hand



D3.1 : State of the Art in Microservice ADL and Microservice Auto-Scaling Strategies

independence of microservices is reduced limiting the gains microservice architecture brings.

- Database per Service: each database is owned by a single microservice. The pattern keeps microservices loosely coupled, preserving team independence. On the other hand queries distributed among multiple databases are made considerably more difficult.
- Querying
 - API Composition: In a context of "database per service" pattern. A specific microservice (e.g. API gateway) queries multiple microservices with dedicated databases to form a composed response.
 - CQRS: In a context of "database per service" pattern. Modifying actions and querying are managed by different microservices. The microservice answering queries has a database that mirrors the modifiable database. The query database can unify multiple microservices dedicated databases, keeping independence intact while alleviating the network load of distributed queries.
- Maintaining Data Consistency:
 - Saga: In a context of "database per service" pattern. For actions that implies multiple microservices, decompose the action in a sequence of smaller actions each local to one microservice. Apply the modifications at each step as long as the local action is successful. The failure of a local action stops the sequence and starts another sequence of compensating actions that undoes the preceding modifications.
 - Event Source: Instead of having objects being the composition of multiple current data states, they are the result of series of events emitted by microservices to an event store. It solves the problem of having events published in the wrong order by saving every past event. A problematic effect of using event sourcing is the request complexity and request load of constructing objects from all past events. Can be used with the SAGA pattern and usually needs the CQRS pattern to alleviate load on the event store.

Application Infrastructure Patterns

Cross-cutting Concerns

- Service Template: Having a simple runnable service whose code is the basis to expand on in order to implement needed functionalities. It serves as a way to speed up production while having with a starting point implementing best practices.
- Microservice Chassis: A framework for microservice development that is an assembly of technologies implementing of the shelf functionalities that are relevant to microservice architecture. This framework makes updating libraries easy and standardized.
- Externalized Configuration: Having all configurations elements dissociated from application by having values stored in configuration files. The benefits are that changing the environment does not imply code modification and separation of business and configurations concerns.

Security : Access Token: In an api-gateway pattern context. Having the gateway manage the initial authentication then passing a token to microservices so that they can verify through a centralized credential store if the request is legitimate (authenticity check and access control).



Deployment : Serverless Deployment: Using a provider that furnishes functions entirely hiding the underlying infrastructure. Can be used as an alternative to microservice architecture, as a particular implementation or in combination with it. It provides some easiness in usage, scaling performance and possibly reduces costs. On the downside it reduces technology freedom, limits greatly infrastructure configuration and can be costly in terms of latency.

Communication Patterns

- Transactional Messaging
 - Transactional Outbox (or application events): To publish an event indicating a change in a database, using an auxiliary co-located database (name outbox) that store the changes. The modification of the database and the outbox are made atomically so published events and database state are assured to be eventually consistent. Two ways of publishing transactions are possible :
 - * Polling Publisher: A service is regularly polling the outbox to publish new transactions to a message broker.
 - * Transactional Log tailing: Another co-located process mines the outbox transaction logs and then publishes changes to the message broker.
- Communication Style :
 - Messaging: Asynchronous communication done through sending messages - documents, commands or events - over channels i.e. abstraction of the infrastructure representing ways to communicate. Although messages can be sent directly from one microservice to another, a common form of messaging is to use a message broker. The message broker is a service that acts as an intermediary, storing messages until recipients consumes it. That way location and availability are not important to the sender.
 - Remote Procedure Invocation: Synchronous, point to point, communication used to invoke functionalities.
- Reliability: Circuit Breaker: In a synchronous communication context. If a number of requests have failed, a timeout is applied during which all incoming requests are rejected.
- External API:
 - API Gateway: Having a microservice being the single endpoint for the entire applications.
 - Backends for Frontends: Having a different microservice as endpoint for each kind of client all build on top a common API layer to interact with the backend.

Observability

- Application Metrics: Services gather metrics during execution. The metrics are then pulled by a server that collects all microservices metrics or pushed by each microservice to the metrics server.
- Distributed Tracing: When treating an external request, assign an id to it that is transmitted to every microservice involved in the request. When producing logs, the microservices save the request id so that the trace of the request throughout the system can be identified.



D3.1 : State of the Art in Microservice ADL and Microservice Auto-Scaling Strategies

- Log Aggregation: Having a service centralizing the logs of every other service in the application.
- Exception Tracking: Having a centralized exception service that receives exceptions from all services as they occur.
- Health Check API: Having services expose a health endpoint indicating different values about its status. A centralizing service invokes all health endpoint to have a global view of the status of the application.
- Log Deployment and Changes: Log every deployment and changes in the application so that errors can be put in relation to them.

Infrastructure Patterns

- Deployment
 - Single Service per Host: The service is deployed on a dedicated host (physical or virtual) ensuring isolation and resource availability at the cost of resource efficiency.
 - Multiple Services per Host: The host (physical or virtual) runs multiple services allowing better resource efficiency at the cost of isolation and resource availability.
 - Service Deployment Platform: Using a deployment solution providing in built functionalities e.g. load balancing, scaling, service discovery...
 - Sidecar: Having another process running alongside a service that handles a particular, often technical, aspect.
 - Service Mesh: Having a sidecar proxy for all microservices instance that manages a range of tasks relative to networking. A centralized control plane component manages the proxies allowing to control network at application level.
- Communication Patterns: Discovery
 - Service Registry: A database of microservices running instances that is queried to allow communications. The service registry is often responsible for performing health checks so that it provides only locations of available microservices instances. Microservices need to be registered in the service discovery and can do so in two ways:
 - * Self Registration: The microservices are responsible for registering themselves in the service registry at startup.
 - * Third Party Registration: A dedicated process or service is responsible for registering the service at startup. While making the architecture more complex, it lightens the code of the application.

Different patterns can be used relatively to how to query the service registry for service discovery:

- * Service-Side Discovery: The service directly queries the service registry to discover the location of a target service. Services host a local registry to limit the network load of querying the service registry.
- * Server-Side Discovery: The service transmit its request to a router that is responsible



for locating and transmitting requests to the target services.

- Server-Side Discovery
- 3rd Party Registration

Those patterns give an idea of what could be expressed in a microservice ADL as they are architectural choices. Being able to identify which ones are being used and which microservices pertain to them would be a way not only to give an understanding of the application logic but also to state and ensure constraints that span over multiple microservices.

2.2 Taxonomy

2.2.1 Classification Criteria

Our ADL overview considers two types of criteria relating respectively to the properties of the language itself and to the provided tools.

ADL Criteria

The criteria used to compare the languages concern *components*, *connectors*, *configurations*, *scalability*, *decomposition* and *focus*.

Components. Concerning *components*, we are interested in the following points:

- **Interface:** The ability to express the points of connections to a component. It can either be a point of connection to the other components or a point of connection to the outside world. In the case of microservices it can be either ports or endpoints.
- **Types:** The ability for a language to classify microservices. It can be in the form of builtin types, furnished by the language, or the ability given to the user to create custom types. The custom types are particularly relevant for functional microservices as they are specific to each microservice application and thus cannot be provided beforehand. The infrastructural type can be subtyped with common infrastructural microservices such as registry, gateway, message broker...
- **Semantics:** The ability for a language to describe the behavior of a language from the outside. It can consist in the description of states indicating the functionalities provided by a microservice depending on previous interaction with it. It can also describe the consequences of an interaction with a microservice on the rest of the system (i.e. analysing, from an initial call on a microservice, the induced calls on its neighbors).
- **Constraints:** The ability for a language to express statements that must be respected. Those constraints can be relative to different domains such as connectivity, response time or implementation. A simple example would be that a particular microservice should not be directly connected to the outside world or that it should be connected to a dedicated database.
- **Implementation:** Guidelines relative to the implementation of a microservice. For instance indicating a required or forbidden technology. It can also be the application of a layer or modification of a property indicating a particular technology. In some cases executable instructions can be specified in the description of a microservice. That category is obviously filled for executable languages.
- **Design patterns:** The ability for a microservice to express that it belongs to a particular design pattern and its role in it. For instance in the "database per service" pattern, the functional



microservice can indicate inline which microservice is its dedicated database and inversely.

- **Datastructures:** The ability to describe the datastructures existing in the application. The most immediate datastructures to be described are the ones associated with functional microservices whose fields are directly involved in the functionalities given by the microservice. Other datastructures such as those transmitted to query the microservices can be specified.

Connectors. We are interested in the presence of a concept of connector which allows to describe the link between components. Connectors are first class entities of a ADL and as such have to be distinguished from the constituents of the interface of components with which it can share some informations. They can be classified using most of the categories used for components.

- **Configuration:** the ability to express architectural structures composed of components and connectors. The most natural structures that can be expressed are graph of components linked with connectors. Doing so a composite is formed providing an abstraction that can be considered and used as a component (Compositionality). The reusability provided by architectural configurations is either the duplication of entirely described parts of applications or patterns where only some fields need filling. Through this approach design patterns can be expressed and abstracted from particular applications, slots for microservices of different natures having different roles can be expected and rules concerning the inner workings of the architectural configuration can be expressed (Constraints). The simple listing of components and connectors admittedly describes precisely the system but does not make sense of it while on the other hand architectural configurations express by essence a superior level of abstraction that highlights the logic of the application. For that reason they can be considered as the concept that makes a language "architectural". Simplicity of syntax, relevance and broadness of application are fundamental for architectural configurations as they mostly determine understandability. However they are difficult to evaluate.
- **Scalability:** The possibility to express rules relative to how the system should scale vertically (i.e. adjusting the amount of ressources allocated to deployed elements) and horizontally (i.e. adjusting the number of microservices instances deployed) depending on system usage. That feature can exist for components, connectors and configurations.
- **Decomposition:** What is the main decomposition of the language, what are the real-world objects that are represented by components and, with a lesser degree of importance, connectors.
- **Focus:** Besides the main decomposition of the language, what is the aspect of microservices applications the languages focuses on.

ADL Toolbox Criteria

The criteria used to classify the tools accompanying ADLs are the following:

Implementation This criteria reflects the possibility to execute the described system from its description. This tool allows to bridge the gap of having on one side the code of the application and on the other its description. Doing so it removes the necessity of keeping the architectural up to date which if not done renders the system description not only useless but potentially harmful. The implementation tool, can be a way of directly executing the system such an interpreter or a way to compile the description to an existing executable target language. In the first case, given that the executional implications of the language is well understood, the execution matches perfectly the description. The downside, apart from the considerable effort needed to produce such a solution, is that, it limits the technology freedom given to developers that is one of the main appeal of microservices architecture. In the second case, the ADL can be technology agnostic and different



code generators can possibly be applied to different part of the system. The maintenance of the tool is then a concern as the generators have to be kept up to date with the target languages. As the architecture language is often much more high level than the target language, the transformation phase has to make considerable choices as to how to realize the description. It is sometimes possible to express guidelines relative to how the system must be implemented in the architectural description of the system. It can go as far as introducing some target language code directly in the architectural description, be a specification phase happening between initial design and generation or a refinement phase happening after generation and modifying the produced code to match the expectations of the developer.

Analysis. The analysis of the system can either be done offline, considering only the documents describing the system, and online, observing the system as it is running. The offline approach tends to check for more theoretical, general, less precise properties of the system since what is actually provable is limited due to the expression capacity of the language, and the specifics of code implementation and infrastructure runtime. Online analysis however can verify that any property is respected, although what it gains in broadness it loses in depth as online analysis has only the ability to ensure that a property has been respected until now but cannot predict with certainty what will happen in the future. For those two, very different but not antagonistic, approaches it is possible to extract specific metrics, measuring a particular aspect of the system through a chosen quantification, or verify given constraints expressed as statements with the concepts furnished by the language or implicitly implied by choices made in the design phase. For instance a constraint expressed explicitly could be forbidding the entrance of outside communication except through one microservice and that exact constraint would be implicitly checked when a "gateway pattern has been used.

Active Specification This criterion indicates whether there is a tool that gives architectural advice based on the current current system descriptions.

IDE Integration This point concerns the existence of a dedicated integrated development (IDE) or modules for existing IDE that provide help for using the language such as syntax highlighting, syntax verification, snippets...

Architecture Reconstruction . This criterion is about a tool that can analyse a running application and produce a description of it in the related ADL.

Visualization . Finally, this aspects indicates whether there is a possibility to produces a visual representation based on the description of the system. The first purpose of this tool is to help understandability of the application. It is particularly useful for text ADLs; it allows to have an idea of the system architecture with a quick glance when reading, connecting and abstracting from the text description can be quite difficult and even error prone for someone not accustomed with a particular application. Even for someone actively working on an application, it helps to have a simpler representation of the system available at all time. A great feature that can be added to that tool is the possibility to represent the system in different levels of abstractions. For instance a level of representation could be the individual microservices, a higher level could be the different communicational patterns and at an even higher connection level the application can be the component represented in its ecosystem. Those different representations, in addition to help developers keeping in mind "the big picture", can fit different kinds of stakeholder. Stakeholders interested only in the business side may not be interested in getting into the microservice decomposition but only in how the application is integrated in a larger ecosystem.



2.2.2 Identified Solutions

LEMMA[25]

is a composition of four languages giving four viewpoints on a microservice application, as follows.

The *service viewpoint* describes the microservices and their interfaces.

The *data structures viewpoint* defines the persistent data and transient data structures. Those data structures can correspond respectively to those contained or linked to the microservices or to the data transmitted between microservices to exploit functionalities exposed through their interfaces. Those two viewpoints are essentially linked together and are the heart of the architectural description proposed by LEMMA. Through the description of data exchange and modification a limited description of business logic is provided.

The *deployment viewpoint* describes the microservices defined in the service viewpoint with the properties relative to deployment.

The last viewpoint is the *technology viewpoint* that allows to define specific implementations technology and specific attributes in need of specification for those technologies. They can then be applied to microservices in the service and deployment viewpoint to give a particular instantiation of the system. The possibility to define transient data structures communicated between microservices and having implementation technology as an additional layer are specific to LEMMA. The main tool provided by LEMMA is implementation generation in JAVA for the development part and with Docker Compose for the deployment part.

```
1 context User{
2   structure User<entity> {
3     string username,
4     string email,
5     string password,
6     immutable int id
7   }
8
9   structure UserService<service> {
10    function User CreateUser(string username, string email, string password)
11  }
12 }
13
14 context API {
15   structure CreateUserRequest<valueObject> {
16     string username,
17     string email,
18     string password
19   }
20
21   structure CreateUserResponse<valueObject> {
22     int userId
23   }
24 }
```

Listing 2: LEMMA Data Modeling Language simplified example

```
1 functional microservice UserService {
2   @endpoints (Spring::_protocols.rest: "/users");
```



```
3 interface Users {
4   @required request
5   @Spring::_aspects.Post
6   create(
7     sync in request : UserDomain::API.CreateUserRequest,
8     sync out response : UserDomain::API.CreateUserResponse
9   );
10 }
11 }
```

Listing 3: LEMMA Service Modeling Language simplified example

```
1 technology Docker {
2   deployment technologies {
3     Docker {
4       operation environment = "docker:25_0_3" default;
5
6       service properties {
7         string image = "openjdk:8u171-jre-alpine"<singleval>;
8         string Dockerfile<mandatory, singleval>;
9       }
10    }
11  }
12 }
13
14
15 technology Spring {
16   protocols {
17     sync rest data formats "application/json" default with format "application/
18       json";
19   }
20
21   service aspects {
22     aspect Post<singleval> for operations {
23       selector(protocol = rest);
24     }
25   }
26 }
27
28 technology MySQL {
29   protocols {
30     sync connectorJ data formats "jdbc_enabled" default with format "
31       jdbc_enabled";
32   }
33
34   infrastructure technologies {
35     MySQL {
36       operation environments = "mysql:5.7.13" default;
37     }
38
39     service properties {
40       string rootPassword<mandatory, singleval>;
41       string mysqlUser<mandatory, singleval>;
42       string mysqlPassword<mandatory, singleval>;
43     }
44   }
45 }
```



Listing 4: LEMMA Technology Modeling Language simplified example

```
1 @technology(Docker)
2 @technology(Spring)
3 container UserServiceContainer deployment technology Docker::_deployment.Docker
   deploys UserService {
4   UserService {
5     Dockerfile = '
6     CMD java -jar user-service.jar
7     '
8
9     basic endpoints {
10      Spring::_protocols.rest: "8080:8080";
11    }
12  }
13 }
14
15 @technology(MySQL)
16 MySQL is MySQL::_infrastructure.MySQL used by nodes UserServiceContainer {
17   default values {
18     rootPassword = "rootpassword"
19     mysqlUser = "mysqluser"
20     mysqlPassword = "mysqlpassword"
21   }
22
23   endpoints {
24     MySQL::_protocols.connectorJ: "3306:3306";
25   }
26 }
```

Listing 5: LEMMA Operation Modeling Language simplified example

Silvera[26]

Silvera[26] is an ADL dedicated to microservices. The description of an application is done through the description of its composing microservices. For each microservice it is possible to give informations about its communication style, its deployment and its interface which is a list of its endpoints. It is possible to indicate dependencies between microservices, one expecting specific endpoints from the other. Some basic builtin typing for microservices is present that matches design patterns provided by the language (api-gateway, message broker and service registry). Silvera proposes a compiler based on code generators plugins converting the described architecture to executable code. By default the implementation is done in JAVA using the Spring boot framework and one selected technology for each builtin infrastructural microservice type. During compilation Silvera's compiler effectuate an analysis of the application and produces through six metrics a qualitative evaluation of the system and of its components.

```
1 service_registry ServiceRegistry {
2   tool=eureka
3   client_mode=False
4   deployment {
5     version="0.1"
6     port=9090
7     url="http://registryexample.com"
```



```
8     host=container
9   }
10 }
11
12 api-gateway EntryGateway {
13   service_registry=ServiceRegistry
14   deployment {
15     version="0.1"
16     port=8080
17     url="http://gatewayexample.com"
18   }
19   communication_style=messaging
20   gateway-for{
21     UserService as /users
22   }
23 }
24
25 service UserService {
26   service_registry=ServiceRegistry
27   communication_style=messaging
28   deployment{
29     version="0.1"
30     port=8080
31     host=container
32   }
33
34   api {
35     typedef User[
36       @id int id
37       @required str username
38       @required str email
39       @required str password
40     ]
41
42     @rest (method=POST)
43     int createUser(str username, str email, str password)
44   }
45 }
46
47 msg-broker Broker {
48   channel EV_USER_CREATED_CHANNEL (UserMsgGroup.UserCreated)
49 }
50
51 msg-pool {
52   group UserMsgGroup [
53     msg UserCreated [
54       str userId
55     ]
56   ]
57 }
```

Listing 6: Silvera simplified example



D3.1 : State of the Art in Microservice ADL and Microservice Auto-Scaling Strategies

is a combination of a microservice ADL (MicroDSL) and a code generator (MicroGenerator) used to describe and generate the code of REST microservice application. A particularity of MicroDSL is that it describes the datastructures associated with the microservices and differentiates between persistent data structures, that should be stored in databases, and non persistent datastructures that exist only for the running time of the microservice and that are lost once execution stops. Microservices stored data, whether they are persistent or not are accessible through predefined REST methods (GET, POST, PUT, DELETE) upon which can be added custom REST methods whose implementation is described inline. While additional information relative to network connectivity can be added, MicroDSL is strongly oriented toward the data viewpoint. From the application description Java code is generated using code generators.

```
1 MicroserviceArchitecture<'Microservice Example Application'>{
2   Microservices:[
3     Microservice <UserService>{
4       Port: 8080
5       AutoDiscovery: true
6       Entity <User> {
7         BaseURL: '/users/'
8         IsPersistable: true
9         EntityResources: [
10          %Single Integer% id [
11            IsIdentifier: true],
12          %Single String% Username,
13          %List String% email,
14          %Single String% password,
15          RESTMethod: [
16            RESTMethodType: POST
17            RESTMethodReturnType: Integer
18            RESTMethodParameters: [
19              %Single String% username,
20              %Single String% email,
21              %Single String% password
22            ]
23            MethodURL: '/create'
24            MethodBody : '/* JAVA code */'
25          ]
26        ]
27      }
28    ]
29  }
30 }
```

Listing 7: MicroBuilder simplified example

MicroArt[28]

takes the opposite approach compared to most of the other microservice ADL. It is firstly thought of as a tool of architecture reconstruction describing both the physical architecture and the logical architecture. To that effect an offline and online analysis of the application is done and its results are injected in the ADL structures. The tool then produces a dependancy graph of the application linking the endpoints of microservices to their callers. After that a manual refinement is done by the user to identify the service discovery microservice and remove its links to others microservices in order to get the logical dependancy graph of the application. The ADL contains typing of microservices, distinguishing between functional and a number of infrastructural microservices. Interfaces



are explicit and a particular entity called "Link", linking a number of sources and target can be seen as a rudimentary connector. The language does not contain a textual concrete syntax.

Jolie[29]

is not a description language but a programming language dedicated to services (though it can be compiled to Java). As such the implementation of the services' functionalities are written in line in a specific section of the service description. The specificity of Jolie is that the microservice application is thought as would be a monolithic application, with the help it provides (e.g. debugging), while keeping the independence of execution intact. For instance, as default behavior of a system written in Jolie, a type mismatch in a request will provoke an exception on the sender side. Another particular capacity of Jolie is that it allows to combine microservices to form new ones through syntactical structures that can be considered architectural configurations. On the deployment side Jolie microservices can be made into Docker images and deployed using Docker or Kubernetes or simply as independent processes. Jocker, a solution using Docker but making it transparent and allowing to use Jolie services contained in Docker containers as if they were not (i.e. as if they were directly deployed on the host) is in development.

```
1 interface UserServiceInterface {
2   RequestResponse: createUser(CreateUserRequest) (CreateUserResponse)
3 }
4
5
6 type CreateUserRequest: void {
7   .username: string,
8   .email: string,
9   .password: string,
10 }
11
12 type CreateUserResponse: void {
13   .userId: int
14 }
15
16 execution { concurrent }
17
18 // MySQL database connection configuration
19 constants {
20   DB_HOST = "localhost",
21   DB_PORT = 3306,
22   DB_USERNAME = "mysqluser",
23   DB_PASSWORD = "mysqlpassword",
24   DB_DATABASE = "userdatabase"
25 }
26
27 embedMySQLClient( DB_HOST, DB_PORT, DB_USERNAME, DB_PASSWORD, DB_DATABASE ) { .
28   conn as databaseConnection }
29
30 service UserService : UserServiceInterface {
31   createUser(request) (response) {
32     atomic {
33       sql = "INSERT INTO users (username, email, password) VALUES (" + "\",\"" +
34         request.username + "\",\"" + request.email + "\",\"" + request.
35         password + "\"";
36       update@databaseConnection(sql) (result);
```



```
34     response.userId = request.id;
35   }
36 }
37 }
38
39 main {
40   //additional service logic
41 }
```

Listing 8: Jolie simplified example

Jhipster[30]

is an application generator that uses a microservice ADL named JDL [31] (JHipster Domain Language) to describe the technology stack and topology of a Microservice application. The application architecture is not flexible as it always consist in a gateway pattern under which functional microservices are connected through a registry pattern. Modifications to the architecture have to be done once the executable files have been generated as JHipster can be considered a bootstrapper providing boilerplate code of interconnected technologies. Beside describing the technology stack, the focus of the language is the definition of the data structures owned by the microservices and their relations. The language only proposes the "database per service" pattern (it is not possible to express the sharing of databases). The language contains only one distinction of microservice type, between functional microservice and gateway. The registry is not explicitly mentioned but created at compile time. A text editor checking the correctness of the syntax is proposed online as well as a plugin for the IntelliJ IDE.

```
1 application {
2   config {
3     baseName gateway,
4     applicationType gateway,
5     packageName com.example.gateway,
6     serviceDiscoveryType eureka,
7     databaseType sql,
8     devDatabaseType h2Disk,
9     prodDatabaseType mysql,
10    buildTool gradle,
11    authenticationType jwt,
12    serverPort 8080,
13  }
14  entities User
15 }
16
17 application {
18   config {
19     baseName userService,
20     applicationType microservice,
21     packageName com.example.userservice,
22     serviceDiscoveryType eureka,
23     databaseType sql,
24     devDatabaseType h2Disk,
25     prodDatabaseType mysql,
26     buildTool gradle,
27     authenticationType jwt,
28     serverPort 8081,
```



```
29 }
30 entities User
31 }
32
33 entity User {
34   id Long required,
35   username String required,
36   email String required,
37   password String required
38 }
39
40 microservice User with userService
```

Listing 9: JHipster Domain Language simplified example

Compose[32]

is a language used to describe and configure "multi-container applications". In that regard it is devoted to a particular instantiation of microservices. The main implementation of the language is its Docker implementation[33]. Being a deployment language, it allows to define some properties of the system relative to the environment in which the container are running such as allocated resources, network connectivity or data storage and properties relative to the environment inside the container such as environment variables or command to be executed at system startup. Through network connectivity Compose describes interfaces of microservices in the form of ports exposed to the others components and of the system with ports exposed to the outside world. If Compose does not have a notion of connector, it is possible to express dependencies between services at deployment time. It is to be noted that while its first purpose is not architecture description, unlike most ADLs it allows to express properties on groups of microservices such as belonging to the same network or shared storage. There is no typing in Compose.

```
1 version: '3.8'
2 services:
3   userService:
4     image: userService
5     environment:
6       - DB_HOST="mysql"
7       - DB_PORT=3306
8       - DB_USER="mysqluser"
9       - DB_PASSWORD="mysqlpassword"
10    depends_on:
11      - mysql
12    expose:
13      - 8081
14
15    gateway:
16      image: gateway
17      depends_on:
18        - userService
19      ports:
20        - "8080:8080"
21
22    mysql:
23      image: mysql:5.7
24      environment:
```



D3.1 : State of the Art in Microservice ADL and Microservice Auto-Scaling Strategies

```
25     - MYSQL_ROOT_PASSWORD: root
26     - MYSQL_DATABASE: userServiceDb
27     - MYSQL_USER="mysqluser"
28     - MYSQL_PASSWORD="mysqlpassword"
29   ports:
30     - "3306:3306"
31   volumes:
32     - ./initdb:/docker-entrypoint-initdb.d
```

Listing 10: Compose simplified example

Kubernetes^[34]

is a system used to configure, deploy and manage containers. Interaction with the system is done through files named *manifests* declaring a state of the application. Kubernetes then either deploy or modify the system so that it matches the description. Kubernetes shares a good part of configurations options contained in Compose but adds its own structures on top of them. Kubernetes more abstract and particular concepts are sometimes substituted to Compose concepts that are relatively close to the real network entities. For instance, opening passage from the outside to a particular service in compose is done through the "Port" keyword in the service description. In Kubernetes a particular object named *service* will act as an intermediary. Kubernetes has the ability to define interior and exterior interfaces with ports but can also manage the routing of endpoints through a specific object called *Ingress*. Request going through the Ingress resource are routed to undetermined instances of a particular microservice. Even though the description of an application through manifests is done using what can be considered as arbitrary concepts, it describes a logical abstraction of the application that is understandable for the user. Because of that it provides a description that is arguably more architectural than some obtained using other ADLs and thus qualify as one.



D3.1 : State of the Art in Microservice ADL and Microservice Auto-Scaling Strategies

```
1 apiVersion: v1
2 kind: Service
3 metadata:
4   name: user-service
5 spec:
6   ports:
7     - port: 8081
8       targetPort: 8081
9   selector:
10    app: user-service
11 ---
12 apiVersion: apps/v1
13 kind: Deployment
14 metadata:
15   name: user-service
16 spec:
17   selector:
18     matchLabels:
19       app: user-service
20   template:
21     metadata:
22       labels:
23         app: user-service
24     spec:
25       containers:
26         - name: user-service
27           image: userService
28           env:
29             - name: DB_HOST
30               value: "mysql"
31             - name: DB_PORT
32               value: "3306"
33             - name: DB_USERNAME
34               value: "mysqluser"
35             - name: DB_PASSWORD
36               value: "mysqlpassword"
37       ports:
38         - containerPort: 8081
39 #
40 #
```

Listing 11: Kubernetes user service simplified example

```
apiVersion: v1
kind: Service
metadata:
  name: mysql
spec:
  ports:
    - port: 3306
  selector:
    app: mysql
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mysql
spec:
  selector:
    matchLabels:
      app: mysql
  strategy:
    type: Recreate
  template:
    metadata:
      labels:
        app: mysql
    spec:
      containers:
        - image: mysql:5.7
          name: mysql
          env:
            - name: MYSQL_ROOT_PASSWORD
              value: "root"
            - name: MYSQL_DATABASE
              value: "userServiceDb"
            - name: MYSQL_USER
              value: "mysqluser"
            - name: MYSQL_PASSWORD
              value: "mysqlpassword"
          ports:
            - containerPort: 3306
            name: mysql
```

Listing 12: Kubernetes user service simplified example

```
1 apiVersion: networking.k8s.io/v1
2 kind: Ingress
3 metadata:
4   name: user-service-ingress
5 spec:
6   rules:
7     - http:
8       paths:
9         - path: /createUser
10           pathType: Prefix
11       backend:
```



```
12 service:
13   name: user-service
14   port:
15     number: 8081
```

Listing 13: Kubernetes simplified Ingress resource

Those different languages have different goals and target different stakeholders:

Language	Goal	Target
LEMMA	code generation	architects
Silvera	code generation	architects
MicroArt	dependencies reconstruction	architects
Jolie	development	developers
MBuilder	code generation	developers
JHipster	technology blueprint	developers
Compose	operation configuration	operators
K8s	operation configuration	operators

Table 1: Principal goals and targets of the existing solutions

2.2.3 ADL Classification

To provide a better understanding of the different properties of the solutions we've identified, we've decided to first analyse the languages in their descriptive features. We use the classification criteria presented in Section ?? . To the exception of "Decomposition"2.2.1 and "Focus"2.2.1, all the criteria are assigned a value of true, indicating the presence of the feature, or false, indicating the absence of the feature. When a feature is part of the language or provided by a tool, it is represented with a green cell containing the letter "Y". Absent features and tools are represented by a red cell containing the letter "N". When it is not clear whether a feature is present, the cell is filled in orange and marked with the "?" symbol.

To ease readability, we split the languages classification table in four parts. A first table tackles the component description features. A second table presents connectors and configurations. A third table reflects other aspects of microservices ADLs. Finally, a fourth table indicates the base concept of the language and what it focuses on.

For all related works, components represent microservices so we use both terms interchangeably.¹

Components

¹ To be more precise Compose and Kubernetes components are respectively containers and pods that refer to the execution environment which encompasses more than just the microservice as a program. However those entities can arguably be considered as a representation of the running microservice.



Language	Interface	Types	Semantics	Constraints	Implementation
LEMMA	Y	~	N	N	Y
Silvera	Y	Y	N	N	N
MicroArt	Y	N	N	N	N
Jolie	Y	N	Y	N	Y
MBuilder	Y	N	N	N	Y
JHipster	N	Y	N	N	Y
Compose	Y	N	N	N	Y
K8s	Y	N	N	N	Y

Table 2: Comparison of components (microservices) description in Microservices ADLs.

Interface. The "interface" column determines if the ADLs explicitly define the interfaces of their components. We immediately notice that it is a shared feature for all the languages but one. Only JHipster does not indicate precise connection points. Connection to and through the database is either generated automatically at compile time using a default configuration or specified through configuration files. The communication between microservices inside the system is generated automatically based on the databases and their relationships that are explicitly indicated in the system description.

The others ADLs allow to explicitly define the microservices' interfaces. All of them allow to determine the exposed port for the microservice which we've considered a sufficient condition to check the interface description box as it is the point of entry into the application while endpoints indicate which functionality is solicited. Despite that indicating the endpoints gives a much more precise vision of what the microservice is through their names, that usually fit the functionalities, and the possibility to establish more sophisticated dependency links.

Apart from JHipster and Compose all the ADLs allow to specify the endpoints. For LEMMA, Silvera, Jolie and MicroBuilder an endpoint signature corresponding to a functionality provided by the microservice is writable inline in the microservice description that indicates the expected parameter and the returned datatype. Component wise Kubernetes only allows to express the port number of the exposed ports of the pod and associated service but a structure called Ingress allows to list the endpoints and their destinations.

Components are the base material of any ADL and interface description being a prerequisite to describe the architecture, it is fairly natural that we observed components' interfaces as an important focus of almost every ADL.

The "typing" column refers to the presence of types of microservices. As discussed in the Criteria Section (2.2.1) the main distinction in microservices types is made between functional and infrastructural microservices. It is possible to expand on the latter with subtypes matching infrastructural microservices commonly found in applications.

As shown by the table it is an uncommon feature of ADLs to provide typing for microservices which can be surprising as it is a distinction that is commonly made when describing microservices applications and that multiple technologies dedicated to a particular aspect of microservice infrastructure are available and widely used. The reasons for the lack of this important feature is that it doesn't align with most ADLs purpose.

LEMMA, Jolie and MicroBuilder are focused on describing the functional microservices, having infrastructural microservices being either implicit and added at compile (MicroBuilder) or added as a supplementary layer (LEMMA). Some of those languages (LEMMA, Jolie) produce deploy-



ment files either in Docker Compose or Kubernetes format so they can rely on the fundamental infrastructural microservices those technologies provide. About functional microservices, they are by essence hard to generalise as they're developed for a specific purpose. It is then reasonable to assume that adding types in those languages seemed irrelevant, useless or even harmful to their creators.

In LEMMA infrastructural microservices can be defined in the "technology viewpoint" and then integrated in the system but only in the "operation viewpoint" limiting their architectural presence in the system description. While it is a de facto type distinction it also sets aside those microservices considering they're somewhat secondary in the architecture relegating them to the technology aspect of things. Because of that and the absence of typing on LEMMA main focus (i.e. functional microservices) we considered the language to be in an intermediate situation relatively to typing.

Compose and Kubernetes are focused on the configuration of the environment. Part of what would be considered infrastructural is already considered as taken care off in the settings they describe (discovery, routing) and for the particular infrastructural microservices used by an application, there is no obvious reason from an operation standpoint to consider them differently than any other microservice. Kubernetes possess the Ingress resource that acts as a gateway but is more akin to a specific microservice than a type.

The languages having a concept of type are Silvera and JHipster. JHipster distinguish only one type from functional microservices which is the "gateway" type. That distinction is driven by the aim of JHipster as a whole that is to produce an advanced prototype of a web application where the user can define the technology stack associated with functional microservices. The application always have a gateway that produces a different code than the other microservices. Silvera also contains a gateway type in which, to the difference of JHipster, the path routing can be specified. Silvera also has a registry type acting as a wrapper for a specific register technology that is specified in a field. MicroArt has no concrete syntax but a metamodel used to make a model of the application through the online analysis of the application. Its metamodel contains a notion of type which is effectively unused so we tend to consider that it does not provide typing for microservices.

In summary, typing is almost non existent in the ADLs we've investigated, the reasons we've identified is the difficulty of determining useful and relevant types for functional microservices, the implicit presence of infrastructural microservices or the absence of a need to identify them as specific microservices.

Semantics. The "semantics" column determines if the language allows for a description of the behavior of the component from the outside (see discussion in the "Classification Criteria" section 2.2.1). To the exception of Jolie, it is a missing feature of all ADLs we've considered. The closest feature to semantics description that some languages provide (LEMMA, Silvera, MicroBuilder) is the indication of the data type transmitted when responding to a specific functionality call. However we regard that as insufficient to affirm that those languages provide semantics description as it belongs to endpoints interface description and that it does not give information to the induced state of the microservice i.e. the change it implies on offered functionalities.

Jolie on the other hand has a specific, simple and appropriate syntax allowing to describe the impact of interaction with the microservice on its subsequent provided functionalities[35]. Using the keywords "provide" and "until" it describes the behavior of the microservice as a machine state.

Compose provides a dependency configuration feature that indicates the order in which the system must be deployed and through online analysis MicroArt is able to express dependencies between endpoints. Those two approach cover some elementary part of what describing the semantics in the system in the sense of describing the communicational impact would be. However we consider



that to properly express that acceptance of semantics it is necessary to establish the link between incoming and outgoing communications. Because of that we consider that no language describes the communicational behavior of microservices. In summary, semantics description is a largely unexplored aspect of microservice applications architecture description. We presume that it comes from the difficulty of establishing an appropriate model and syntax for it but also because most of the ADLs we've looked at are oriented toward code generation and thus leave that aspect of microservice applications to their concrete implementation. We consider that as a deficiency as semantics are a considerable part of what makes a language architectural. Moreover, in the context of code generation, semantics description could make the generated much closer to the architect intent thus reducing the manual refinement process. Jolie being a dedicated microservice programming language doesn't have that discrepancy and pertinently cover that aspect.

Constraints. The "constraints" field determine if it is possible to express statements that microservices must respect. Coupled with the appropriate tool, it allows the architect to be sure that its component description does not break those general rules. It is also a way to ensure that it won't happen in future modifications of the system and even if there is no verification it has the merit of explicitly stating the imperatives relative to a particular component.

We consider that a language would be able to describe constraints if it can do so in a direct manner and not as a consequence of another description. For instance LEMMA allows to specify the protocol of an endpoint. We consider this as a description of the endpoint and not a proper case of constraint that should be expressed for the whole component. For instance a statement establishing that all endpoints of a component must be used only with a particular protocol would be a valid case of constraint.

While those concerns are properly architectural, we did not find a way to express them independently in any of the languages we've considered. We see that as a deficiency as constraints give a proper architectural information on components. Some simple constraints could be easily expressed such as a rules on the number of connections, on usable technologies, on communication protocols, on position in larger configurations or on the execution environment. Executional constraints such as response time or connection limit are also possible to express and couple with monitoring tools.

Implementation. The "implementation" column refers to the ability to give indications about how the described component should translate into a program. For the languages that provide such a possibility, we must distinguish two categories.

The first category consist of the executable languages. Indeed Jolie, Compose (through Docker Compose) and Kubernetes have an execution tool that allows to immediately execute the system. Obviously the implementation category becomes either irrelevant or validated by default for them. We can note, that even if executables, the expressive power of the language sometimes excess the executional power of the related execution tool. For instance, it is possible to express a number of replica in Compose while the Docker Compose platform does not have a mean to use that information.

The second category are the languages that allow to associate technologies with components. In LEMMA it is possible to define technologies in a particular viewpoint that can be placed as an additional layer upon described microservices. This applied technology then implies some precisions to be added to the microservice description for the description to be correct and the code generation to be performed. Other technologies, such as infrastructural microservices or databases can also be defined in the technology viewpoint files and then only referred to in the deployment section of the application. JHipster does not let the user define custom technologies but includes them in the description of microservices. Some arguably infrastructural responsibilities (e.g. data



storage, authentication...) are then taken care of by resulting microservices invoked at execution time.

The last category includes the languages that allow to directly write target code which is obviously some sort of implementation indication. MicroBuilder demands the user to precise the Java code of non implicitly generated methods and LEMMA in its deployment section allow the user to introduce some Docker code. The aim of an architectural language is to provide an higher level view of the system and since procedural code often needs deep analysis to understand its ramifications its introduction in architectural description is not to be generally considered desirable. However in the case of those languages that are directed toward code generation and use the introduction of executable code in a limited and well-defined manner it limits the discrepancy between the architectural description and the target executable code that is generated. Moreover those precisions limit the ulterior refinement work.

Silvera does not includes concepts dedicated to implementation and MicroArt being used as a language to only describe existing architectures has no need for it.

In summary, we've seen that most languages are either directly executable or provide some way of giving implementations directive. The fact that this area is well covered is to be put in touch with the fact that most of these languages aim to be used for code generation.

Connectors and Configurations

Language	Connectors	Configurations
LEMMA	N	N
Silvera	N	N
MicroArt	Y	N
Jolie	N	Y
MBuilder	N	N
JHipster	N	N
Compose	N	Y
K8s	N	Y

Table 3: Presence of connectors and architectural configurations in microservice ADLs.

Connectors and configurations are considered essential parts of an ADL (see ??) however they are absent from most ADLs or barely represented.

Connectors. The "connector" column indicates whether the language has a notion of connector or not. As discussed before, interfaces are to be distinguished from connectors as they define the points on which the connectors are meant to connect to. Through that specification they can give limitations about which kind of connector can be used but connectors must be independent entities representing a communicational link between two components. Because of that, explicit dependencies written in components descriptions are also not connectors. In a fully and well described system they would imply the existence of a connector that could render their specification unnecessary.

That being clarified we now consider the presence of the connector concept in existing ADLs and remark that it is almost non-existent. Only MicroArt provides a rudimentary concept of connector under the name of "Link" that is created by offline and online analysis of the system and consisting only in the source and destination of the connection. As MicroArt does not have a concrete syntax



it serves only as an intermediary to produce the dependency graph of the application.

Jolie has a concept name *Aggregator*, discussed in the next paragraph, that comprises some parts of what a connector is such as having input and output port corresponding to other microservices as well as protocol specification. However a Jolie aggregator does much more than what a connector is expected to do such as protocol transformation, and unified response construction from different microservices. It also manifest itself as an independent process at runtime and so we felt that it did not fit into the connector category that is meant to represent immaterial relations between components.

Connectors not being addressed is a important deficiency in existing ADLs as without them the description of the application tends to be a collection of unorganized elements and thus miss the exhibition of the architecture.

Configurations. The "configuration" column refers to the possibility to express architectural configurations in the language. As it is the case with connectors configurations are an uncommon feature of microservices ADLs.

LEMMA, Silvera, MicroBuilder and Jhipster don't have concepts that can be considered configurations. MicroArt has a notion of "cluster" in its metamodel but makes no use of it as no information extracted from the offline or online analysis of the application is mapped to the concept. We then considered that it did not provide architectural configurations.

Jolie has two concepts that can be considered architectural configurations: aggregation and embedding. Aggregation is used to orchestrate groups of services that produce different responses and produce a single composed response to the caller. It describes a composition of microservices but has an existence of its own, both logically and at runtime as an independent process, as it performs actions that are not limited to the piping of microservices communications. However its primary function is to organise the workflow of existing microservices describing, the logic tying them together and can thus provides a higher level of abstraction in which the aggregator is seen as a component to its immediate environment. Because of that we consider aggregators to be architectural configurations.

Embedding is a concept in Jolie that allows to combine existing services into a new, bigger one. Embedded services are then parts of a larger service and not existing anymore as independent processes but as part of a new service that usually add unifying logic on top of them. That part can make it arguable that embedding is not an architectural configuration since it does more than describing the system. We consider that if we dissociate the language from its implementation (i.e. current interpreter) the added logic can be seen as a description of what's happening in its parts as the embedding concept can be used as a wrapper around the contained services. Because of that the embedding concept is not only fitted to be considered a configuration but also the more advanced example of configuration we've encountered as it allows to define a custom logic for services interactions.

Compose proposes a series of keywords that affect the entirety of the system. The "network" keyword creates a communicational space shared by default by all the services of the system or by a selected number of them that register to it in their description. Multiple networks can be defined creating communicational cluster of services. In the same manner, the "volume" keyword define a cluster of services sharing a data storage location. That ability of clustering given by Compose makes it so that it can be considered as featuring builtin specific architectural configurations.

Kubernetes offers resources such as Ingress, Services, Deployments and Pods can be seen as architectural configurations as they allow to cluster services. Ingress allows to route incoming commu-



nications toward different kinds of pods depending on the path, it thus defines a cluster of pods accessible from the outside of the application. Deployments are usually used to manage one kind of pod but can refer to multiple pods. Services create a communicational cluster for different pods. Pods can be constituted of multiple pods, in practice usually a main service and its dedicated services such as a database or an initializer. Doing so it produces a composite.

Surprisingly enough, the executable languages, whose main purpose is not the description of the architecture, are those that contain architectural configurations that are arguably the highest degree of architecture description. That is partly explainable by the fact that architectural configurations are essential for those languages to perform while it appears as superfluous to other ADLs. We consider that as a deficiency, even more than we did with connectors, since the architectural configurations are a condition to make sense of the system in a global manner regardless of the viewpoint (e.g. communicational, logic, resources...).

Design Patterns, Data Structures and Deployment Features

Table 4 reflects three aspects of microservice systems that can be frequently found in ADLs.

Language	Design Patterns	Data Structures	Deployment
LEMMA	N	Y	Y
Silvera	Y	N	Y
MicroArt	N	N	N
Jolie	N	N	~
MBuilder	Y	Y	N
JHipster	Y	Y	Y
Compose	N	N	Y
K8s	N	N	Y

Table 4: Presence of other microservices systems aspects in microservice ADLs

Design Patterns. This column indicates whether or not design patterns appear in one form or another in the language.

Compose and Kubernetes do not have a notion of design pattern although some of their structures suppose implicitly the presence of tools whose functioning implements some of the more usual patterns. The "network" concept in Compose and the "service" concept in Kubernetes for example imply the presence of the "registry pattern" and of the presence of service discovery through DNS. However these languages do not refer explicitly to those patterns which is what is expected of an architecture language representing them. The Ingress component of Kubernetes can be seen as acting like a gateway but describes rather a microservice than the architectural impact of a gateway pattern as it doesn't encompass microservices indirectly accessible. Lastly if the Pod structure allows to create some topologies which correspond to design pattern (e.g. database per service), it does so implicitly, by having a description that happens to correspond to the pattern but is not identifiable as such in itself and is no reusable.

Jolie happens to fall in the same category as its topological structures are not parametrizable enough to express a design pattern in different applications. It is possible to express a particular topology in an application that would fit a given design pattern but it is necessary to include all the possible services that can be used and apply conditionals to determine which one of them is used in this particular instance of the composite structure.



LEMMA and MicroArt simply don't have structures or keywords to indicate the usage of a design pattern.

Silvera, MicroBuilder and JHipster all provide references to well known design patterns in their components' descriptions. In Silvera, when describing a microservice you have to explicitly reference its registry, implementing de facto the "registry pattern". MicroBuilder and JHipster both couple databases with functional microservices implementing de facto the "database per service" pattern but do not give the option to opt for a "shared database" pattern.

In summary design pattern are not widely used in existing ADLs for description of microservice applications. Most of the languages use design patterns implicitly in the sense that their toolboxes either generate applications with fixed patterns or that the tools used to execute the code, in the case of executable languages, rely on them. That situation can be seen as pragmatic but is also a loss in generality as it limits the range of architectures those languages can describe. The best situation would be to dispose of structures allowing for the creation of reusable, parametrizable architectural configurations whose particular forms would match design patterns. Unfortunately no existing ADL provide such a concept in its concrete syntax.

Data Structures. This column indicates if a language allows to describe the data structures existing in the application. As Compose and Kubernetes are not involved with the functional code, they don't have any mean of describing data structures linked with microservices.

Jolie possess a series of inbuilt method that allow to communicate with a number of database technologies in a standardized way but do not allow to define persistent data structures owned by microservices. Being a programming language it allows for "runtime" data structures to be defined in the microservices code.

LEMMA, MicroBuilder and JHipster have a strong focus on data structures used in the system and are the only languages we studied that allow to explicitly define them. All three languages allow to define databases owned by microservices de facto implementing the "database per service pattern" and not giving the possibility to describe architectures with shared databases.

Microbuilder does it in the most straightforward manner by simply having the data structures defined in microservices' descriptions. It provides a distinction between data structures stored in an independent database whose persistence does not depend on the microservices life as a program and data structures that exist for the lifetime of the microservice and that are lost when it stops its execution.

JHipster defines data structures separately from microservices. The microservices can refer to a number of data structures. Each microservice referring to a data structure has to be considered as having its own instance of the data structure, effectively existing as a dedicated database whose particular technology implementation is determined in the description of the microservice. A specificity of JHipster regarding databases is that it allows to describe relationships between databases.

LEMMA is the language where the data concern is the most developed. Data structures are described in their own section of the language named "data viewpoint". The main distinction is made using the domain-driven design of "entities" and "value object" who represent respectively persistent data structures and transient data structures. The data structures' definitions can then be used in the microservice viewpoint. Entities are typically objects in databases dedicated to a microservice which implements the "database per service" pattern. Where LEMMA stands out is in allowing to define precisely through value objects the volatile data structures used for requesting and answering requests. Through its data viewpoint, LEMMA address the data communication concern in the most precise way we've encountered.



In summary, there are different examples of data structure representation in microservices' ADLs focussed on defining the databases dedicated to specific microservices. While it is not as specific to microservices as the "database per service" pattern is, "shared database" pattern is not specifically describable in any of the languages which is a limitation in terms of architectural description.

Deployment indicates whether or not the language allows to insert descriptions relative to the deployment of the system.

Compose and Kubernetes are runtime environment configuration oriented so they do this by design.

Jolie and Microbuilder don't have a deployment section anywhere in the architecture description. The closest indication relative to deployment is that in both of those language the exposed port is specified in the microservice description. Jolie adds the location (url or ip address) and couple deployment configuration files to the main system description.

JHipster and Silvera both have a "deployment" keyword used in the scope of the architecture description to parametrize the deployment. JHipster uses either Docker Compose, Kubernetes or Open Shift for deployment and it uses its "deployment" description to choose between those tree technologies and configure them using specific fields for each of them. Some technology agnostic keywords such as "number of replicas", deciding between "ephemeral" or "persistent" storage allow to indicate some general notions about how the system should be deployed. In JHipster multiple deployment blocks, for different technologies, can be defined at the architecture description root, with each of them ultimately producing deployment for the whole system. Silvera can only produce a Docker Compose files at generation time but defines the deployment block inside microservices description, allowing to have specific instructions for each of them. It expresses the deployment in a more technology agnostic manner making no references to the particular technology it uses in the deployment block. The most determining fields that can be defined are "port", "replicas" and "restart policy". It is to be noted that it is possible to specify in the "host" field if the application is intended to run in a "physical machine, virtual machine or container".

LEMMA as for the data concern, dedicates a whole viewpoint to the deployment concern named "operation viewpoint" that is a part of the architecture description expressed in dedicated files. Services are imported either from the "service viewpoint" for functional microservices, or defined directly as infrastructural services using technologies defined in the "technology viewpoint" files. Each of these kinds of deployment units specify exposed ports and use a deployment technology defined in the "technology viewpoint" files that allows to specify specific fields to fill (e.g. image, dockerfile). The infrastructural deployment units have specific fields allowing to express on which infrastructural nodes they're depending and which functional services depend on them. They also permit to specify some technology specific fields (e.g. database password).

In summary, set aside deployment configuration languages, deployment is a common feature in microservices ADLs but stays on the surface leaving the specificities to be modified in the necessarily existing or generated deployment files.

Decomposition and Focus



Language	Decomposition	Focus
LEMMA	microservice	interface, data
Silvera	microservice	interface
MicroArt	microservice	interface, dependencies
Jolie	microservice	interface, logic
MBuilder	microservice	interface, data
JHipster	microservice	data, technology
Compose	container	configuration
K8s	pod	configuration

Table 5: Main decomposition and focus of microservice ADLs

Decomposition represents the kind of component the different ADLs use to decompose the application and the focus column indicates the main axis of description of that component. Unsurprisingly in most of the cases the base system brick is the microservice. Here we distinguish microservice from container or pod, respectively the unit of decomposition for Compose and Kubernetes.

Microservice designate the program and in all cases but JHipster, the microservice is first described through its interface and without any technology specification. Under that aspect of component description which is necessary to any architectural description, different focuses are brought by the different languages. Data description provided by LEMMA, MicroBuilder and JHipster is a key aspect of microservices as it poses specific challenges to preserve their independence. Jolie being a programming language has as a main focus to describe the microservice logic which is entirely contained in its description.

JHipster is a particular case as it does not explicitly provide endpoints. The "application" structure, that in a microservice context refers to a microservice, acts as a blueprint, gathering and configuring different technologies taking part in the final program. We considered that it is core to JHipster as it constitutes the microservice description. On the other hand in LEMMA the technology model apply as an overlay to the technology agnostic service description.

Focus. While to some extent the container and the program executing can both coincide as being the "microservice" the distinction is to be made when considering the focus of a language. Indeed Compose and Kubernetes use as base element not the program but the environment in which it is executed. In accordance with that decomposition the description sets the properties of the inside of the components, where the program is just a particular attribute among other such a environment variables, and the edges of the components with properties such as ports or allocated resources.

2.2.4 ADL Tools Classification

Toolboxes are fundamentally linked to the languages that describe an architecture, often they are the reason the language is created in the first place. It is particularly obvious for Compose (through Docker Compose, its main implementation) and Kubernetes for which the language and supporting tool are so perfectly aligned that in practice users make little difference between the two.

We distinguish between tools that are relative to implementation, to analysis and to visualization.

Tools for ADL Execution

Table 6 presents the tools that are relative to the execution of ADLs.



Language	Executable	Generation
LEMMA	N	Y
Silvera	N	Y
MicroArt	N	N
Jolie	Y	Y
MBuilder	N	Y
JHipster	N	Y
Compose	Y	Y
K8s	Y	N

Table 6: Presence of implementation tools for identified solutions

Executable refers to the existence of a tool to directly run described architecture without compiling it to an existing, executable, target language. Three languages fit that categorization: Jolie, Compose and Kubernetes.

Jolie is a procedural programming language dedicated to create microservices systems and so it comes with a specific interpreter that has been realized to run instructions (e.g. parallelism) or producing behavior (e.g. message transmission error) designed to be particularly relevant for microservices architectures. Whether the system is run using a container technology or the jolie command, each service is a different process running a different instance of the Jolie interpreter. The Jolie interpreter is written in Java so it relies on the Java Virtual Machine to run.

Compose files are executable with the docker-compose tool that translate those declarative files in a docker application that meets the described properties. In the end the tool associated with the compose description is then docker and from a point of view compose acts as a wrapper around docker, avoiding the pains of scripting or command typing to obtain the desired system. As compose relies ultimately on Docker, it benefits from all its features such as service discovery, network communication and isolation or security features. It also adds functionalities on the system such as scaling.

Kubernetes' files, known as manifests, are intended to be used to interact with a Kubernetes running system that manages resources, taking away the infrastructural concern of deciding where and how to set up the modifications, from the user. Setting up the Kubernetes system has a relatively high initial cost but once its done the infrastructure is made completely abstract. What is left for the user to do is simply to describe what it desires and transmit it to the Kubernetes interface that will take the appropriate measures to make so that the real state of the running application match the description. Kubernetes is often referred as an orchestrator, which is appropriate if we think of orchestration as dealing "with how different services are composed into a coherent whole"[36]. Kubernetes fit that broad and blurry definition by taking charge of a multiplicity of strong requirements for a microservice application such as service discovery, scaling (both vertical and horizontal), message routing and many other aspects. It also fit the more precise definition of an orchestrator specifying that "in particular, it specifies the order in which the services are invoked, and the conditions under which a certain service may or may not be invoked"[36]. It is to be precised though that Kubernetes does take charge of that aspect from the infrastructural point of view, for instance through load-balancing and routing, but does not get into the control flow of application logic which was historically the initial subject of orchestration. However, to be effective the described Kubernetes system description must match the application logic.

Generation refers to whether or not the corresponding microservice ADL is coupled with a tool that can generate code in a language that already disposes of the tools needed for it to be executable



(e.g. an interpreter, a compiler to machine code).

MicroArt and Kubernetes don't have a tool for generating code in an executable target language. MicroArt is intended to be used in architecture reconstruction and provide information about microservices dependencies in the system; converting the extracted information in an existing executable language would have been a totally different purpose and challenge. Kubernetes relies heavily on its own specific structures and is designed to be coupled to its particular platform which makes it difficult to translate. Also it is fairly recent and arguably still the last major deployment and operation technology that has arisen so the need for translation might not exist yet.

Jolie and Compose are executable languages (meaning that they dispose of a tool to directly run the code) but can also be translated respectively to Java and Kubernetes. Jolie furnishes a tool named "jolie2java" that permits to convert a Jolie service to Java. Jolie furnishes Java libraries to allow easy compatibility between Jolie and Java services, when using jolie2java, the Java services produced are then able to directly invoke the Jolie services' operations. Compose has a tool named "Kompose"[37] used to convert Docker Compose files to container orchestrators, and in particular to Kubernetes. It is an official Kubernetes project. It generally requires little modification to the produced files to have a working application and allows to complete the Docker Compose file with custom labels to obtain the desired output and limit manual refinement. It is to be note that as Kubernetes implements more functionalities than Docker Compose, some microservices might have been initially designed to take care of task that are native to Kubernetes and thus resulting in a non optimal architecture where specific Kubernetes resources are not taken advantage of and even acting as redundant parts of the system.

LEMMA, Silvera, MicroBuilder and JHipster are mainly oriented toward code generation. They all provide means to generate Java code either through a command line compiler tool (Silvera, MicroBuilder, JHipster) or as an Eclipse plugin (LEMMA). Silvera, JHipster generate code using templates in a model-to-text transformation, while MicroBuilder compiler named "MicroGenerator" transform the system description in a direct model-to-text transformation. To clarify, the description of the system is the model that is transformed to text, that transformation is done either directly or disposes of a modular component, a template, that will determine through place holders replaced at compile time what the final code will be. LEMMA has an additional initial step to this process, converting the system description in instances of builtin metamodels (mapped on the different viewpoints in the language) that can then be used in a model to model transformation (effectively toward a general object oriented language metamodel) before being transformed to code. That supplementary step allows for a lot more flexibility and possibilities such as model analysis and permitting the creation of transformations tools to other programming paradigms metamodels. Ultimately in each of these solutions, a refinement phase is need after code generation.

Available Microservice ADL Tools

Table 7 represent the architecture analysis tools available for the different languages we studied.



Language	Offline		Online	
	Constraints	Metrics	Constraints	Metrics
LEMMA	N	N	N	N
Silvera	N	Y	N	N
MicroArt	N	N	N	~
Jolie	N	N	N	N
MBuilder	N	N	N	N
JHipster	N	N	N	~
Compose	N	N	N	~
K8s	N	N	N	~

Table 7: Existing architectural analysis tools

Silvera is the only language having a tool to establish an offline evaluation of the system; it does so by calculating metrics at compile time, in a phase preceding code generation. The six metrics evaluate mainly the independence of microservices in order to draw attention on potentially critical or problematic parts of the system. Each microservice is subject to evaluation for each metric, this process resulting in an application score (i.e. average of all microservices scores) for a particular metric.

MicroArt through online analysis is able to extract microservices dependencies. However it does not provide a comprehensive way of presenting the gathered info, leaving the real analysis part and deduction making to the user.

Online analysis is relatively unexploited. Kubernetes natively produces metrics about resources used by pods on the underlying infrastructure hosting them, resources used by the kubernetes system itself and custom metrics exported by applications. In the same manner, Docker produces resources metrics accessible through the API. Since those platforms control the execution of the application it is not surprising that they produce metrics about resource usage. However these raw metrics are not correlated to the architecture of the microservice application and thus do not produce an evaluation of the system.

A surprising observation is that, apart from JHipster, generation oriented ADL do not automatically integrate technologies to produce metrics. JHipster at compile time integrates components to produce resources usage metrics (Prometheus), ways to exports application specific metrics (Logstash) and to produce a graphical representation (Grafana).

We note that a recent article proposes a method to analyse LEMMA operation models to detect antipatterns and correct them[38]. At the time of writing it has not been integrated to the language.

Analysis is a particularly untouched territory which is explainable by different factors. Before thinking about having a tool to verify constraints it would be desirable to dispose of ways to express constraints about a microservice system. If such an expression capacity was existing in a language, matching this syntax with an existing, general, model analysing tool (e.g. Alloy[alloy_website]) could prove less tedious than one could anticipate.



Language	Representation	Abstraction	Views
LEMMA	N	N	N
Silvera	N	N	N
MicroArt	Y	N	N
Jolie	N	N	N
MBuilder	N	N	N
JHipster	Y	N	N
Compose	N	N	N
K8s	N	N	N

Table 8: Existing visualization tools

Visualization is the most effective tool to represent high level informations. It is useful to share informations between different domains, but also for domains specialists to have an easy reminder of the system and its environment.

Surprisingly, it is a feature of only two of the identified solutions. MicroArt is oriented toward architecture reconstruction and uses a graphical representation to represent the system it analysed. In the representation, microservices are represented as a collection of boxes, one for each identified endpoint with a section for exposed endpoints and another for required microservices. Those boxes are connected with lines indicating a dependency. As MicroArt does not provide a concrete syntax, it is fair to say that the graphical representation is the actual language and not a graphical representation of it. MicroArt does not provide different levels of abstractions or views of different aspects of the system.

JHipster provide a very simple representation of the different microservices as boxes containing the different "entities" and their relations. The visualization is generated directly as the description is being written and even though it is rudimentary it shows the power of a graphical representation as local modifications appear immediately and clearly in their relation with other preexisting elements. The visualization provided by JHipster could be considered a view as it describes only the data structures. Through its IntelliJ plugin, JHipster also generates UML representation.

The usefulness of those two examples shows that visual representation is a missing feature in ADLs that could be an important and relatively easy tool to integrate. Producing a mapping from the ADLs to existing technologies, such as the C4 model[39] that already provides a way to represent different abstractions level, could be an effective way to do so.

Additional Tools

Language	IDE	Active Specification	Architecture Reconstruction
LEMMA	Y	N	N
Silvera	N	N	N
MicroArt	N	N	Y
Jolie	Y	N	N
MBuilder	N	N	N
JHipster	Y	N	N
Compose	N	N	N
K8s	N	N	N

Table 9: Other ADL tools



IDE if a microservice language either provides a dedicated IDE or plugins for existing IDEs. The most usual way to provide IDE support for ADLs is to furnish a plugin for an existing IDE. The languages that do so are JHipster (IntelliJ), LEMMA (Eclipse) and Jolie (Visual Studio Code) with all of them providing syntax highlighting and text completion. JHipster is the only language to propose a dedicated IDE accessible through its website.

Active specification is not provided since static analysis for microservices is almost non-existent. Most of the systems we've identified make strong decisions that can be considered as good practice for the type of application they aim but it cannot be considered active specification as it does not adapt to the particular architecture being described. Positive active specification is particularly challenging as it needs not only to understand the described architecture but also know of the relevant architectural compositions through either an exhaustive list or general rules. The tool must then be able to correctly identify an unsatisfying architecture and a better proposition to present the user with.

Active specification can be seen as the pinnacle of user support, but its implementation as a positive proposition is difficult to achieve. An easier implementation would be the detection of anti-patterns and the proposition of a solution. That tool is not provided by any of the solutions we identified but an article presenting a proof of concept using LEMMA to identify and correct architectural anti-patterns has recently been published[38].

Architecture reconstruction is the main focus of MicroArt which focuses particularly on endpoints dependencies through static and online analysis. MicroArt architecture reconstruction tool first analyses the source code then performs an online analysis to ultimately ask the user for a refinement phase where the service discovery microservice is identified manually and removed. As it acts as an intermediary the tool links the identified calls to the service discovery microservice to the corresponding calls from the service discovery microservice and reveal the logical dependencies in the system.

MicroArt code[[MicroARTGithub](#)] is not usable anymore but other techniques have been developed to reconstruct microservice architecture. As for MicroArt, it is a bit of a stretch to call them languages as their concrete manifestation is a graph of boxes and lines that is usually purely descriptive and cannot be used to model new systems or modify existing ones. Different techniques have been explored in academic papers to reconstruct the architecture such as log collection, packet interception and code analysis. It is also a large area of interest in the industry due to the ever-growing complexity of microservice applications[40]. However they focus on the particular concern of microservice dependencies representation and are not coupled with a language allowing to describe the system in its different aspects.

2.3 Research Challenges

The current state of the art of Microservices ADLs is oriented describing the components rather than the architecture linking them together. That is observable through the predominance of code generators in the toolboxes and the large absence of architectural configurations in the lexicon of microservices ADLs. As a consequence of that lack in the capacity of expression, analysis is also underexploited and visualization almost nonexistent. It appears enriching the existing languages with concepts aiming to reveal the architectural structure of microservices systems is still a challenge to address. It would open fields of investigations that are close to critical aspects of microservice architecture. Through the analysis of existing microservices ADLs, we've identified several research opportunities.

Defining syntactic constructs to describe MSA architectures is one of these opportunities. Not only



that it would by itself make the microservices systems more understandable but it would allow for other interesting means of expressions and corresponding tools. Indeed, existing solutions target runnable code and are not oriented toward describing the underlying architectural structures and patterns. That is obvious by the lack of structures to describe architectural configurations. When such structures exist they do in a very pragmatic way. They are either tied to fundamental necessities of an application execution environment (Compose) or are specific to the tools to which the language is destined (Kubernetes). Jolie is an exception but the architectural configurations structure they provide are thought as executional units that relegates engulfed services as parts of a new bigger service, adding logic instead of describing the one that is already present. If reusable they also lack the possibility to be instantiated with different microservices than the one they were created from.

The most immediate usage would be to use architectural configurations to create builtin compositions corresponding to well known design patterns instantiable with particular microservices. The possibility given to users to define their own structures would allow to cover yet unknown developments of microservice architecture.

Coupled with the development of a language for expressing constraints, which is presently another missing item from existing microservices ADLs, the underlying logic of patterns could first be expressed but also verified given that the necessary tools are developed or that the translation to existing verifying tools is made. Those expressed rules could also translate in online analysis tool to check in real time if the expected properties are verified.

The existence of a customizable superstructure to the microservices, would be a mean to produce views of the system. Those views could be levels of abstractions by looking at composites without presenting the details of their constituting microservices (e.g. a functional microservice and its dedicated database) but also in larger systems abstracting from complex patterns to only look at the big picture. In a textual representation that mechanism would be as simple as folding a class in object oriented development to hide the methods but it would also be a mean to couple that language expression with a graphical representation tool or a translation to an existing one.

In addition to the expression of communicational patterns having constructs to group microservices can be a way to reorganise microservices around concerns giving means to users to focus on particular aspects of the system. In the current state of microservice descriptions concerns are scattered among multiples, if not all, microservices in the system description. That situation has a lot of drawbacks in terms of understandability, redundancy and even coherence. Above all it is error prone as correct description or modifications of the system relies on the user going through the complete document to make adjustments in multiple places.

Connectors become less essential to architecture description if architectural configurations exist. They could even be thought of as particular instances of architectural components, as they are also immaterial relation between components.

The most important challenge for such a syntactic structure to exist is the possibility to use it on existing systems. Having a distinct language for architecture description is not a satisfying solution as it adds an important and constant workload in the application development. It would increase the risk of discrepancy between architecture description files and the actual state of the application leading to misrepresentation. This problem has been solved by all the ADLs we've studied mainly by making the language executable or at least a basis for code generation.

Executable languages are however confronted with the problem of adoption. Languages used to develop microservices architectures now have mature ecosystems that are well known which



makes the adoption of new technologies difficult. The absence of technology lock-in in microservice development can alleviate that difficulty allowing the usage of a new language for only parts of the system but then the architectural reach of the system description is then greatly limited. Because of that it appears that the usefulness of an architectural language is conditioned by its ability not only to be converted in an executable program, which is a solvable engineering problem, but by its ability to be the target of conversion from existing systems. The research challenge will then lie in the possibility to extract meaningful structure through static and dynamic analysis.



3 Microservice Auto-Scaling Strategies

Before focusing on microservice scaling (Section 3.2), let us introduce the background on *scaling*.

3.1 Background

This section presents some important definitions related to scaling before detailing scaling strategies in the general cloud context.

3.1.1 Autonomic Computing, Self-optimisation, and Auto-Scaling

Elasticity is one of the key features in cloud computing that dynamically adjusts the amount of allocated resources to meet changes in workload demands [41]. *Scaling* is the process used to provide elasticity, it corresponds to the action of adapting the computational resources allocated to a system.

Auto-scaling is the property of a system to automatically detect the need of and perform scaling [42]. It is a specific aspect of *Self-optimisation* which is at the core of *Autonomic Computing*. As introduced in 2003, autonomic computing is provided when computing systems can manage themselves given high-level objectives from administrators [43]. Self-optimisation concerns the system's capacity to identify and seize opportunities to make themselves more efficient in performance or cost.

Self-optimisation is achieved through the control loop defined in the MAPE-K architecture [44].

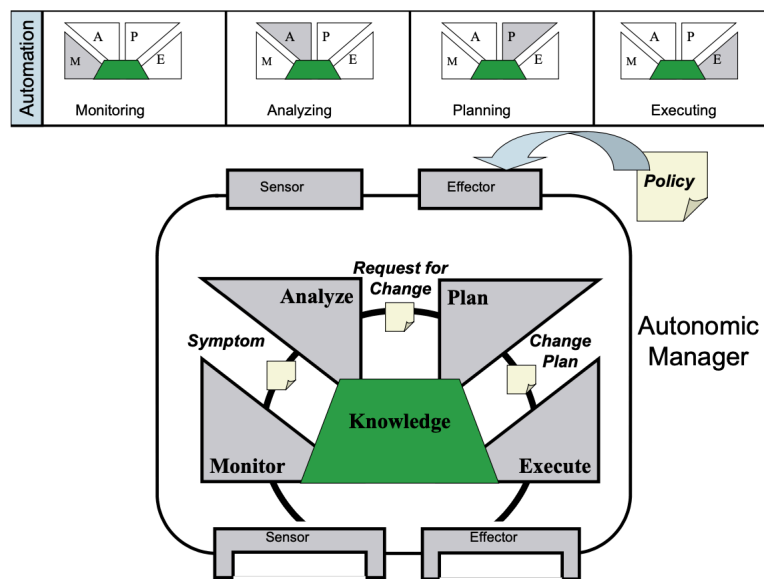


Figure 3.4: MAPE-K Architecture from [44]

3.1.2 Elasticity in Cloud Computing Taxonomy

Elasticity has been explored extensively by researchers from academia and industry fields. In their paper [42], Al-Dhuraibi et al. provide an extended classification for the elasticity mechanisms according to the configuration, the scope of the solution, purpose, mode, method, etc. They propose the taxonomy presented in Figure ??

This taxonomy distinguished seven main aspects, namely:

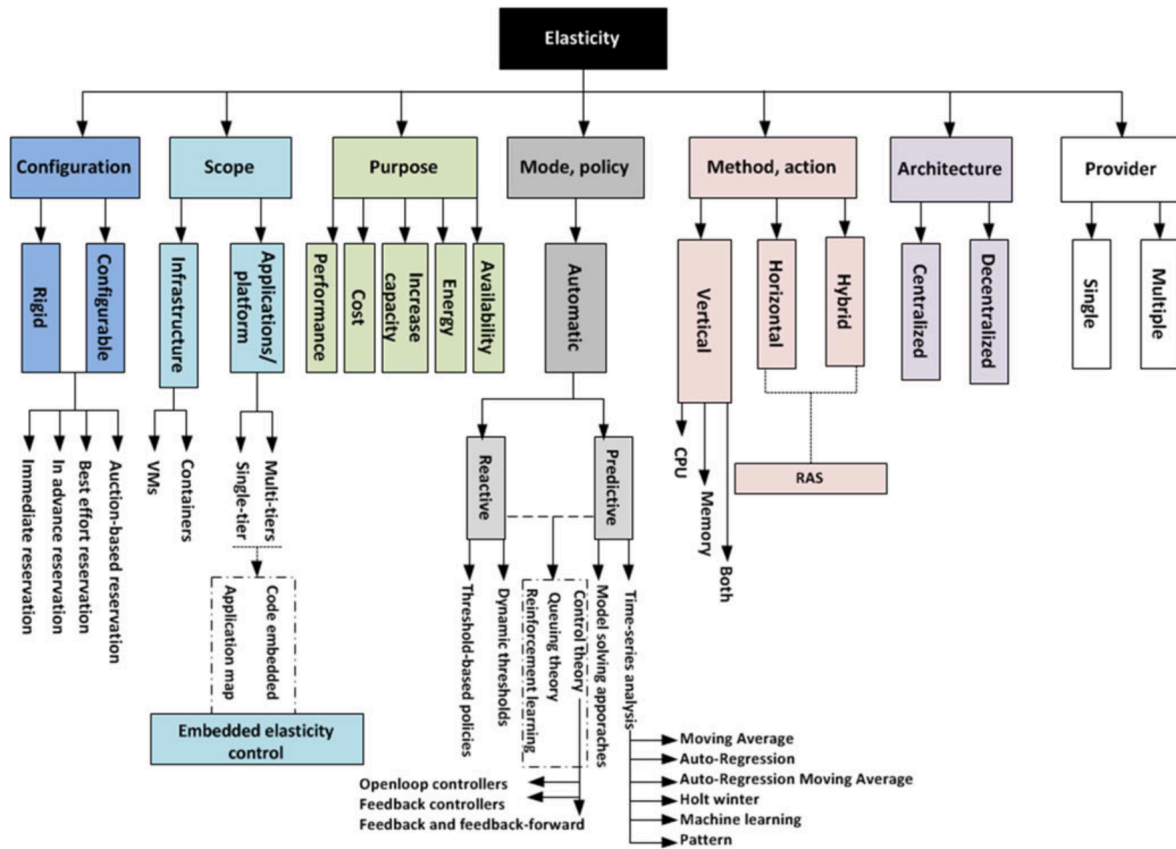


Figure 3.5: Elasticity Taxonomy from [42]

- *Configuration*: represents the method of the first or initial reservation of resources with a cloud provider and may be fixed or adaptable.
- *Scope*: defines at which level, infrastructure or application/platform, elasticity actions are applied.
- *Purpose*: one of improving performance, increasing resource capacity, saving energy, reducing cost and ensuring availability.
- *Mode, policy*: refers to the needed interactions in order to perform elasticity actions.
- *Method, action*: distinguishes vertical, horizontal and hybrid scaling.
- *Architecture*: distinguishes between centralized and distributed solutions.
- *Provider*: Elasticity may concern single or multi-provider environments.

3.2 Microservice Auto-Scaling Strategies

3.2.1 Background

Microservices are defined in terms of functionality encapsulation and are thus software architecture entities. However, as they are to be deployed and managed independently, microservice instances are execution entities. The running support for microservices are *processes* (cf. Section 2.1.3) which in most cases are encapsulated in virtual machines (VMs) [45, 46] or more commonly containers [47,



[48] . Microservice auto-scaling has therefore naturally inherited the auto-scaling mechanisms and strategies introduced for VMs or containers.

3.2.2 State of the Art on Microservice Auto-Scaling

There are several recent review papers on microservice auto-scaling [49–53].

[49] is a general introduction to the major concepts of microservice scaling. The concepts that are presented are the ones borrowed from the cloud ad in particular from containers. The authors follow the concepts presented in 3.1.2 and classify existing works according to the *type* and the *strategy* of scaling. The type corresponds to one of horizontal, vertical and hybrid scaling, while the strategy may be proactive or reactive. The metrics monitored for scaling are mainly CPU and memory usage, the supported load in requests per second (req/s) and the response time.

[50] focuses on the relation between load balancing and auto-scaling for containerized microservices. The authors put forward the fact that load balancing directly impacts and helps auto-scaling. However, most related works consider separately the two subjects. The interesting conclusion is that in most cases optimization strategies target a single objective (minimize response time, optimize resource cost, maintain performance, etc.) and that considering jointly multiple criteria is a hot issue.

General Classification In [52], the initial target of the authors is to investigate proposed solutions and research directions for the formal verification of microservice auto-scaling. As this particular question is quite new to the domain of microservices, the authors orient their study towards solutions proposed for the cloud. However, they present a well-structured, comprehensive and quite exhaustive vision of the SOA on microservice auto-scaling. A synthetic view of their taxonomy is presented in 3.6.

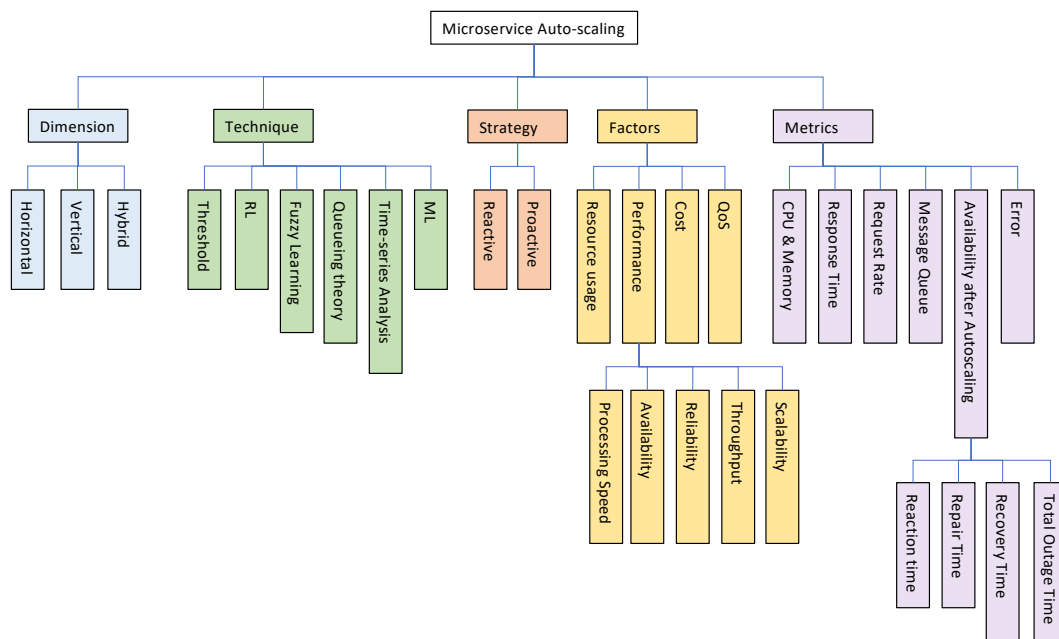


Figure 3.6: Auto-scaling Classification Taxonomy from [52]

- **Dimension = Scale what ?** The dimension criterion answers the question of how resources are allocated during microservice scaling. As in the classical cloud context, microservices may



be scaled vertically, horizontally or by combining both approaches. Vertical scaling (up and down) implies the adaptation of the quantity of resources allocated to a single microservice instance. Horizontal scaling operates by adapting the number of replicas (instances) for a microservice [54, 55].

- **Factors = Why scale ?** Auto-scaling is used to maintain quality guarantees for the microservice system. These guarantees mainly concern four use cases. A first case of interest is the resource usage of the system where the goal is to minimize the used resources and consequently minimize exploitation costs. A second major issue is performance where scaling may typically be used to minimize processing time, maximize throughput or guarantee a minimal time between failures. The system cost in terms of renting costs (strongly connected to the quantity of resources and their energy consumption) is a third aspect to consider. Finally, higher-level, e.g. application, quality of service requirements are of prime importance.
- **Technique = To scale or not to scale ?** In relation to the target factors of the microservice system, different approaches are used to detect the need for auto-scaling. The simplest and widely provided strategy is to use thresholds. Going over an upper threshold for a given resource will trigger the increase of resources while going below a threshold will trigger resource liberation. A different strategy is to use time-series analysis on the execution traces of the system. The idea is to detect behavioral patterns and be able to detect them and react in an appropriate way. Currently, because of the scale and the complex architectures of microservice systems, the most used techniques for identifying the need for scaling are ML-oriented, presented later in this section.
- **Strategy = When to scale ?** Microservices follow the standard proactive or reactive scaling strategies. Reactive scaling is triggered after a problem in the system behavior has been detected, while proactive scaling aims at preventing the problem.
- **Metrics = What is the important system state ?** As described in the MAPE-K loop, an auto-scaling solution needs to monitor the system to be able to feed the decision techniques and decide whether scaling is needed or not. The used metrics are closely related to the scaling factors and are discussed in the SCALER Deliverable D1.1.

[53] proposes a recent, not yet published, extended review on microservice scaling. The authors define more fine grained classification by introducing new criteria. These are:

- **Elasticity scope.** This aspect focuses on the system level concerned by the elasticity actions. The authors differentiate between the application level and the infrastructure level. This characteristic refines the previously introduced notion of **dimension**.
- **Elasticity purpose.** The authors explicit two important aspects of elasticity, namely time efficiency and resource optimization. Time efficiency characterizes the reactivity of the scaling process. Resource optimization concerns the effort to minimize the allocated resources. These are
- **Elasticity triggers.** The authors detail more the different methods used to trigger elasticity, which is an elaboration of the **Technique** feature.

ML-Oriented Classification [51] tackles the issue of container orchestration. Indeed, cloud service providers have to handle hundreds or thousands of containers simultaneously and require a refined and robust container orchestration system to control overall resource utilization, energy efficiency, and application performance. The paper tackles orchestration applied to different architectures and aiming at different objectives. Microservices are one of the considered architectures and auto-scaling is filed under the category of methods for resource provisioning. The identified



objectives are resource efficiency, energy efficiency, cost efficiency and SLA assurance.

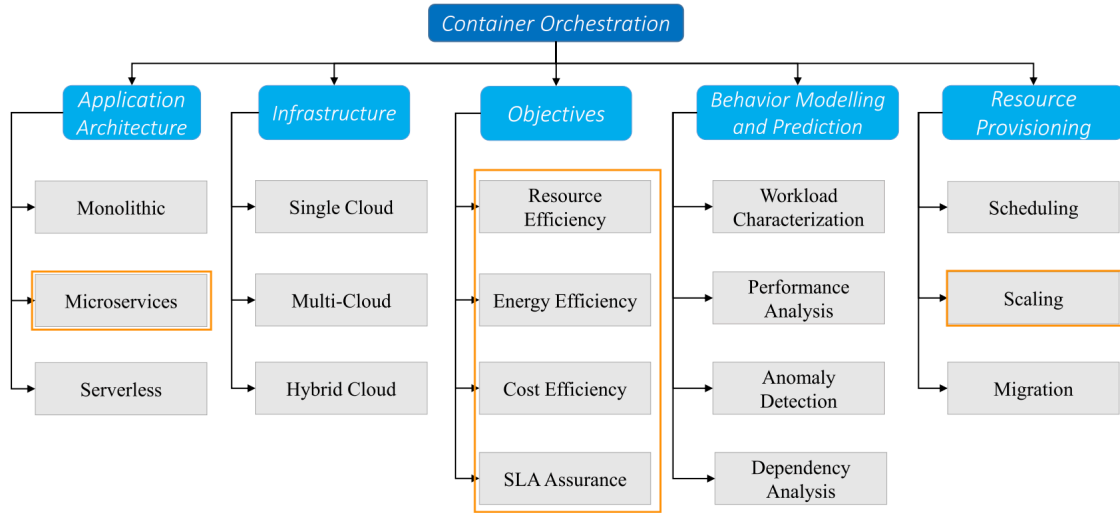


Figure 3.7: Auto-scaling Classification Taxonomy from [51]

The primary focus of the paper is the Machine Learning (ML) approach to orchestration of containers. The authors retrace the history of using ML methods for orchestration and observe that there is a limited number of methods and models (cf. Figure 3.8).

Following the classification shown in Figure 3.7, the authors classify related works. In the following, we extract the works that relate to our subject of interest i.e. microservice scaling, and complete with the additional criterion of the execution environment the approach has been proposed into. Indeed, as the SCALER project ambitions the enrichment of the K8S ecosystem, we investigate whether the proposed solutions are or could possibly be integrated within K8S or not.

More in detail, our methodology is the following:

- We have considered all the classification aspects that relate to scaling. Indeed, the overview identifies several major domains of work, including *Workload Characterization*, *Performance Analysis*, *Anomaly Detection* and *Dependency Analysis* (cf. the taxonomy given in Figure 3.8) above). In all four cases, scaling may be involved as a way of achieving the target service level objectives (SLO).
- For all cited works, we have considered four additional criteria. These are namely, *K8S*, *Scaling*, *Year* and *Validation*. The first aspect reflects whether the work has been done using the K8S orchestrator. The *Scaling* criterion indicates whether scaling has been considered in the work. The *Year* simply points out how recent the work is. Finally, *Validation* indicates whether the work has been validated through simulation or using a real deployment environment. Another interesting point, to be further discussed, concerning the validation is the considered scale, both of the application and the underlying infrastructure.
- Once this first pass on related works done, we have focused on works that have been done in the context of K8S and consider scaling. We have crossed out the references between the different classification tables (cf. Appendix A) to check to which domain the *Scaling*-related works belong. For example, there are works targeting anomaly detection and then applying scaling for recovery.
- Finally, we have reconsidered all the papers from the scaling classification to position them vis-à-vis the goals of SCALER.



D3.1 : State of the Art in Microservice ADL and Microservice Auto-Scaling Strategies

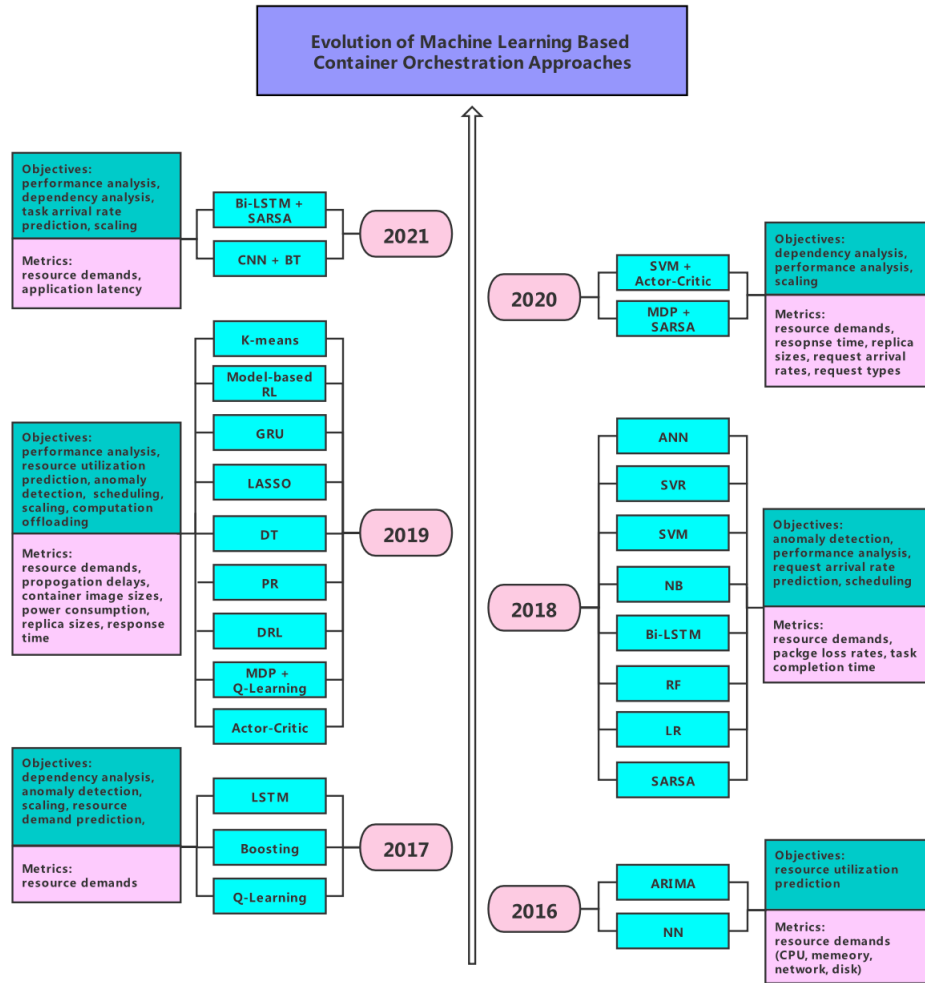


Figure 3.8: ML Orchestration Techniques from [51]

Here is our analysis of the papers that are the most relevant to SCALER.

Microscaler [56] (reference [65] in the initial paper) proposes a scaling solution focusing on service latency. The authors propose a novel metric, *service power*, defined as the ratio between the average latency of the 50% of the slowest requests over the average latency of the 10% of the slowest requests. To track the latency, *Microscaler* uses the functionalities of the Istio service mesh. To decide of the number of replicas to consider they use Bayesian Optimisation (BO). The system is validated over a medium to small scale system: it considers an application composed of 10 microservices on a self-constructed distributed system of 10 VMs. The work is interesting for SCALER as it explores the idea of monitoring microservice interactions and succeeds it with an acceptable overhead. Moreover, the authors propose an interesting metric while targeting one of the Key Performance Indicators (KPI) as identified in our Deliverable 4.1, namely response time, moreover with the same quality requirements (between 100 and 300ms). The question remains however whether this approach can be adapted to a larger scale.

The work presented in [57] (reference [93] in the initial paper) trains a forest tree model to learn the relation between four metrics and the number of containers (instances) needed to guarantee the desired QoS. The four metrics are total access, concurrency, average response time of users and error rate. The first two characterize the load pressure, while the latter characterize the QoS. It



seems that the authors need to produce an important volume of data to train the model and it is unclear at what scale this is done (the paper mentions variations of 1 to 20 containers but this supposes only one microservice). Their choice of metrics seems however relevant for SCALER.

RScale [58] (reference [66] in the initial paper) models the scaling problem as a constrained optimization problem resolved using Gaussian Process regression. The problem is solved using data points from multi-level system observation (metrics from the container and the VM level) and ensures respecting a target service tail latency. The scale of the validation is medium as it is done with a 10-microservice application on 4 servers with 16 VMs. The drawback of the approach is, as indicated in the overview paper, its high computational complexity and thus overhead. Indeed, the approach builds and makes evolve a model per microservice, which implies that the constraint problem is to be periodically solved also per microservice. However this work is interesting with its multi-level observation metrics as well as with its capacity to tackle variations in multitenant environments.

[59] (reference [67] in the initial paper) establishes resource profiles for pods to guide its proactive scaling strategy. To construct the profiles, the authors test the target applications beforehand and establish a relation between the served request rate (requests per second) and the needed pods. The scaling decision is done choosing a forecast between three alternatives from three different ML methods. The validation is done in a real K8S deployment with one master and five workers. The idea of combining multiple ML models and dynamically choosing the proactive scaling action on the basis of an accuracy estimation is quite interesting and is to be considered in SCALER. However, in the proposed implementation, the target applications are encapsulated in a single pod which is a major simplification making the proposed profile computation unfeasible in real scenarios.

[60] (reference [74] in the initial paper) is to be considered as it is the only work focusing on vertical scaling. The authors investigate the performance analysis of co-localized containerized applications to decide of resource needs and thus vertical scaling. However, the approach is a preliminary as it uses brute force for exploring configurations in a well-defined case considering three specific applications. The authors explore different learning prediction models that relate available features (resource limits, request rates, SLIs of other applications observed simultaneously) to SLIs but these models are specific to the set of considered applications and once a model is chosen it is applied throughout the execution of the MAPE-K loop. This work relates to *Performance Analysis*.

HANSEL [61] (reference [78] in the initial paper) proposes a hybrid scaling approach composing both reactive and proactive mechanisms. It constructs a model for workload prediction based on real-world traces. and thus relates to *Workload Characterization*. The workload prediction model is used in a reinforcement learning model for a proactive scaling decision. The RL model decides upon the monitored load, the predicted load, the resource usage and microservice response time. The reactive mechanism enhances the default K8S autoscaler by combining the logic of resource usage monitoring with monitoring of the microservice response time and comparing it the the service level requirement. If the methods proposed in the paper are of great interest to SCALER, the validation of the scaling mechanism considers a small cluster of two workers and a small (deprecated) microservice-based application. Moreover, even if 5G is a major motivation of the paper, the actual consideration of its characteristics are left for future work.

Sinan [62] (reference [23] in the initial paper) relates to *Dependency Analysis*. It trains two models to predict respectively short-term latency and long-term QoS violation probability. The first model is built using the microservice graph and thus accounts for microservice dependencies. The paper also contributes to the optimized exploration of the search space to get data to train the models. Sinan has been validated using two of the DeathStarBench applications on a 4-server cluster: the scale is thus medium to large. Sinan is of interest to SCALER for three major reasons, namely 1)



it is a very recent work showing the need of analysing microservice dependencies 2) it allows for model retraining and thus takes into account dynamic environment changes and 3) it considers group scaling where groups are defined in terms of resource allocations. Indeed, Sinan considers the scaling of a single microservice, of k microservices with the least resource usage, as well as of all microservices.

FIRM [63] (reference [24] in the initial paper) relates to *Dependency Analysis* as it localizes the microservice instances responsible for SLO violations by extracting critical paths and detecting anomaly instances. FIRM uses resource, workload, and performance metrics to make dynamic provisioning decisions that may include either horizontal or vertical scaling. It shows that it succeeds in improving by 10 to 30 times the behavior (mitigation of latency degradation through scaling) of the target DeathStarBench applications compared to other solutions including K8S autoscaling.

MIRAS [64] (reference [85] in the initial paper) is not listed in the scaling classification table of the overview while it does consider scaling. The system considers microservice-based applications that represent scientific workflows. What is to be taken away for SCALER from the paper is the approach which proposes a model-driven RL method with a low sample complexity for resource adaptation in the cloud.

3.2.3 Research Challenges

In the following we discuss open research issues and their relation to SCALER's motivations.

Microservice Application Scale All related works and microservice-oriented reviews agree to put forward the challenge of microservice applications' scale. Indeed, modern architectures can typically count dozens of different microservices which give birth to numerous microservice instances interconnected in complex execution graphs. Scale is thus a major challenge to intelligent scaling which will need to find their way and pinpoint the microservices that need to be reconfigured.

Auto-Scaling Performance Scaling mechanisms should to be able to handle efficiently important volumes of observation data and quickly decide which microservices need to be considered for scaling. In addition, scaling is to be done with minimal overhead to prevent request loss and QoS degradation.

[51] puts forward the need for characterizing the workload distribution across the system, as opposed to modeling and prediction of the request arrival rates and resource usage patterns of individual microservices, which support the SCALER's intuition about identifying target microservice groups.

Microservice Application Heterogeneity A major challenge of microservice applications is their implementation heterogeneity. Indeed, microservices are a design principle which does not impose the use of a particular language, framework or any other technology. If gRPC and REST HTTP protocols are widely used especially in e-commerce applications, microservice interactions take various forms and may involve many other complex communication schemes. The question of the design and implementation of a *generic* scaling mechanism is thus still an open question. In relation to this observation, [51] identifies the need for *systematic* anomaly detection and recovery under different application architectures and cloud infrastructures.

Multi-target Auto-scaling Scaling may be required to pursue QoS requirements related to different KPIs. Typical examples are tail latency, downtime, processing throughput and resource usage cost. Most of related works focus on a single QoS requirement and recent approaches consider one requirement in conjunction with cost factors. Future intelligent scaling mechanisms are to be able to consider multiple KPI constraints and allow for the configuration of their respective weights. This observation is supported by [51] which puts forward the need for multi-dimensional perfor-



mance optimization to manage the tradeoff between different performance metrics, especially SLA violations and financial costs.

Loose Coupling vs Efficient Auto-scaling It is easy to control and to reconfigure a target system when the system's architecture and deployment structure are explicitly known in advance. Indeed, with such *static* configurations, the needed observation information, the frequency at which it needs to be scraped and processed, as well as the optimized detection of hot spots are relatively easy to establish. However, microservice-based systems are the exact opposite of such well-defined static architectures. As microservices are loosely-coupled, the microservice interconnection graph is only known by the developer teams, and when the scale is really important even they may have only partial views. The system's architecture is only partially reflected in its deployment configuration files (cf. Section 2 on microservice ADLs) and additional knowledge is difficult to reverse engineer. The fact is that the available explicit information on the target system is limited. As a consequence, there is a tension between two goals: the need for a clear architecture picture for efficient scaling, and the practice of a flexible dynamic large-scale architectural style for microservices. In this line of thought, [51] puts forward the need for microservice dependency analysis to explicitly address the relationship between the status of individual microservices and overall application performance metrics.

3.3 Cloud-Native Network Function Auto-Scaling Strategies

3.3.1 Background

The concept of "Cloud-native" or CNF (Cloud-native Network Function) network functions is gaining in importance. A network function is a software entity that performs specific tasks related to the management, control and provision of network services like for example (Enhanced Mobile Broadband (eMBB) or ultra-reliable low latency communications (URLLC)). "Cloud-native" network functions represent a new generation of network functions that take advantage of the remarkable advances in containerization technologies. The main idea is to implement scalable, self-repair, progressive and flexible network functions by exploiting the principles of "native cloud". This includes breaking down the operators network functions into multiple light and interdependent software components called micro-services. Each micro-service is encapsulated in a container.

3.3.2 State of the Art on Cloud-Native Network Function and Microservice Auto-Scaling

Various scaling solutions have been proposed in the state of the art described in this section.

In DeepScaling [65], an approach to maintain a stable CPU usage level around a desired level is proposed. In this perspective, the authors have identified three challenges. The first, given that a cloud-native architecture introduces complex interactions that are difficult to extract, making the task of predicting traffic load tedious. For this, the authors propose a deep learning algorithm called Spatio-temporal graph neural networks (STGNN) which both takes the temporal aspect and also spatial (between the different micro-services) in order to predict traffic in a more efficient way which allows a wider decision window to react to a scaling event. The second challenge is to estimate the CPU usage that will be generated by the traffic load, the authors have designed a model based on a deep probabilistic network as a processor utilization estimator, to accurately estimate the processor consumption, even under special conditions, such as periodic tasks and internal system events. The final challenge is to estimate with precision the resources required for each service based on the estimated CPU usage, which is of course the ultimate goal of automatic scaling. The relationship between required microservices and CPU usage is generally complex and non-linear, especially when service resources are at critical usage levels. To solve this problem, the authors proposed an improved Deep Q Network (DQN) model with a loss function based on the



target usage level, which allows to quickly find the optimal resource need.

In [66], the authors highlight the challenges of deploying a cloud-native 5G Core Network (CN), which is designed to be dynamically managed and oriented towards micro-services. For this, they develop AI-based schemes for autoscaling network functions, specifically focusing on the Access and Mobility Function (AMF). The proposed approach is an AI module based on a neural network architecture (CNN with its Dilated CNN version [14]), uses metrics from the CN to forecast demand and make scaling decisions, aiming to admit a higher number of users to the 5G network and balance traffic among AMF replicas. Also, the article compares the proposed AI-based approach to traditional autoscaling methods, demonstrating that the AI solution can lead to more stable network operation with fewer instances of network functions deployed, due to better resource utilization.

In [67], the authors address the challenge of managing load and scalability in the 5G Core network, focusing on the Access and Mobility Management Function (AMF). They propose a control theory-based algorithm to balance the load across AMF instances and to dynamically adjust the number of AMF instances (scaling in or out) based on network load. This aims to maintain optimal response times and reduce processing latency while saving energy and avoiding resource wastage.

In [68], a proactive approach to dynamically scale the 5G core network's Virtual Network Functions (VNFs) using Recurrent Neural Networks (RNNs), is presented. The authors propose a machine learning-based strategy that forecasts upcoming network load and schedules resource provisioning accordingly. They compare their RNN-based approach with a traditional threshold-based scaling technique and a DNN-based approach. The results show that the RNN-based solution outperforms the threshold-based method by proactively scaling resources before load increases, thus avoiding AMF congestion and reducing the time required to process user requests. The RNN-based solution also slightly outperforms the DNN-based solution in terms of prediction accuracy and resource utilization.

[69] proposes a new method to dynamically adjust the core network resources of 5G, in particular for the access and mobility management (AMF) function. This method uses machine learning (ML) techniques to predict traffic load and anticipate resource requirements. The prediction of future traffic loads and the required quantity of AMFs is determined using two distinct neural network methodologies: the Deep Neural Network (DNN) and the Long Short-Term Memory Recurrent Networks (LSTM). Simulation results show that the forecasting method is more efficient than solutions based on static thresholds, because it reacts faster to traffic changes and reduces the time it takes to get new resources ready for use. The proposed approach is proactive and avoids the delays associated with setting up new instances of VNF (virtual network functions) when increasing traffic load.

[70], presents solutions to improve the scalability of mobile networks in response to the rapid increase in mobile traffic. The authors propose two predictive horizontal auto-scaling models for both containerized and non-containerized versions of the Evolved Packet Core (EPC) in cellular networks, utilizing the built-in features of Amazon Web Services (AWS). The authors propose an efficient task assignment scheme for AWS aimed at maximizing throughput and ensuring fairness among competing instances. Two solutions are introduced: a Relaxed Optimized Solution (ROS) and a Heuristic Approach (HA). Experimental results show that the container-based model is more resource-efficient compared to the VM-based model. The ROS and HA solutions also improve throughput and fairness compared to existing solutions.

[71] presents two main contributions to the field of Network Function Virtualization (NFV) at the network edges. Firstly, the authors propose a neural network model to predict the required number



of Virtual Network Function (VNF) instances based on network traffic. The model, trained on traffic traces from a commercial mobile network, achieves a higher prediction accuracy. Secondly, they introduce an Integer Linear Programming (ILP) formulation for the optimal placement of VNFs at edge nodes. The primary goal is to minimize end-to-end latency for users accessing these functions. The results show a significant improvement in latency. The authors also emphasize the importance of proactive auto-scaling and dynamic VNF placement in NFV infrastructures to meet the demands of next-generation network services.

3.3.3 Research Challenges

There are several main challenges facing 5G-network application scaling, namely:

1. An unprecedented application scale : 5G networks target much higher data flows which in turn will reflect in the services and interactions in the 5G application.
2. Lack of 5G-core application implementation : R&D on G scaling is hindered by the lack of a full functional implementation of the 5G-core application.
3. Diversity in network functions and a stateful/stateless strain : Numerous papers consider a single type of network functions to scale while there are dozens of them. A full scaling approach will need to consider the 5G-core architecture at its full complexity. Moreover, many network functions are stateful which is an aspect which excludes automating horizontal scaling.
4. Strong dependency on real-life infrastructure and traffic and its inherent confidentiality issues: R&D results are to be validated in a realistic setting which requires the manipulation of real-world mobile user data which in most cases is confidential.



4 Concluding Remarks

In this report we have presented an overview of Microservice Architecture description languages (ADLs), as well as an overview of microservice auto-scaling solutions. This study has validated the major intuitions that have motivated the SCALER project. Namely:

Microservice-oriented ADL. There are very few and very partial solutions for describing microservice architectures. An expressive microservice-oriented ADL is an open challenge that can bring enormous benefits in terms of project management and performance.

Microservice Auto-Scaling. Even if an impressive number of auto-scaling solutions have been proposed with the expansion of the cloud usage, microservices bring new performance and complexity challenges that ask for innovative smart auto-scaling solutions.



A Classification of ML-Oriented Works [51]

In all following tables the indicated references are the ones from the initial paper.

Figure A.9 gives the augmented classification of works on *Workload Characterization*.

Ref.	Mechanism	Infrastructure	Application Architecture	Methods	Objectives	Advantages	Limitations	K8S	Scaling	Year	Real/simu/
[18]	Regression	Single cloud	Microservice	TNNR	Resource demand prediction	Improved prediction accuracy	Limited workload scenarios	×	×	2016	real
[70]	Regression	Single cloud	Microservice	GBR	Resource demand prediction	High prediction accuracy	Limited workload scenarios	×	✓	2017	simu
[81]	Time series analysis	Single cloud	Microservice	LSTM	Request arrival rate prediction	Low time overhead	High computational expense	×	✓	2019	simu
[82]	Time series analysis	Hybrid cloud	Microservice	IGRU-SD	Resource demand prediction	Low error rates	Unclear demonstration of the relationship between resource allocation and energy efficiency	×	×	2020	simu, Google traces
[67]	Time series analysis	Single cloud	Microservice	AR, LSTM, HTM	Request arrival rate prediction	Multi-model optimization	High computational costs	✓	✓	2020	simu + AWS
[71]	Time series analysis	Single cloud	Microservice	Bi-LSTM	Prediction of request arrival rate and application throughput	Improved prediction accuracy	Implicit time overhead analysis	traces	×	2018	real+simu
[78]	Time series analysis	Hybrid cloud	Microservice	Bi-LSTM	Task arrival rate prediction	Performance improvement	Inaccuracy in long-term forecasts	✓	✓	2021	real

Figure A.9: Works on Workload Characterization of Microservices from [51].



D3.1 : State of the Art in Microservice ADL and Microservice Auto-Scaling Strategies

Figure A.10 gives the augmented classification of works on *Performance Analysis*.

Ref.	Mechanism	Infrastructure	Application Architecture	Methods	Objectives	Advantages	Limitations	K8S	Scaling	Year	Real/simu/
[85]	Regression	Single cloud	Microservice	ANN	Performance modeling of microservice workflow systems	Sample complexity reduction	Randomness due to boundary effects	✓	✓	2019	real
[86]	Regression	Multi-cloud	Microservice	SVR, ANN	Performance modeling of microservice response time	Time saving for route searching	Dependency on offline training of historical data	✓	✗	2020	real
[87]	Regression	Hybrid cloud	Microservice	LR, PR, RF, and SVR	Prediction of offloading execution time	High prediction accuracy	High computational costs	✗	✗	2019	real
[66]	Regression	Single cloud	Microservice	GP	Prediction of end-to-end latency	High prediction accuracy	High computational complexity	✓	✓	2020	real
[74]	Regression	Single cloud	Microservice	LR, RF, and LASSO	Prediction of SLI	High prediction accuracy by removing anomalies	High computational complexity	✓	✓	2019	real
[23]	Time series analysis, regression	Single cloud	Microservice	CNN, BT	End-to-end latency and QoS violations prediction	Online prediction with high resource efficiency	Overfitting and misprediction	✗	✓	2021	real
[75]	Regression	Single cloud	Microservice	DT, MARS, boosting, and RF	Modeling and prediction of SSD I/O performance	Short data training time	Simplicity of application models	✗	✗	2021	real

Figure A.10: Works on Performance Analysis of Microservices from [51].



Figure A.11 gives the augmented classification of works on *Anomaly Detection*.

Ref.	Mechanism	Infrastructure	Application Architecture	Methods	Objectives	Advantages	Limitations	K8S	Scaling	Year	Real/simu/ real
[42]	Classification	Single cloud	Microservice	KNN, SVM, NB, and RF	Anomaly detection of microservices according to real-time performance metrics	Various monitoring metrics	Insufficient evaluation of application-level anomalies	✓	✗	2018	real
[69]	Time series analysis	Single cloud	Microservice	LSTM	Anomaly detection and prediction on application or infrastructure-level metrics	Improved prediction accuracy	Static models	✗	✗	2017	real

Figure A.11: Works on Anomaly Detection of Microservices from [51].



Figure A.12 gives the augmented classification of works on *Dependency Analysis*.

Ref.	Mechanism	Infrastructure	Application Architecture	Methods	Objectives	Advantages	Limitations	K8S	Scaling	Year	Real/simu/
[86]	Regression	Multi-cloud	Microservice	BO, GP	Discovery of optimal route of individual microservice	Load balance	Poor performance under highly dynamic environment	✓	✗	2020	real
[24]	Classification	Single cloud	Microservice	SVM	Identification of potential heavy-loaded microservice units	High accuracy	Implicit explanation of the time overhead and computational costs	✓	✓	2020	real
[23]	Classification	Single cloud	Microservice	CNN	Dependency analysis of microservices between pipeline-tiers	Navigation of system complexity	Overfitting and misprediction	✗	✗	2017	simu
[69]	Time series analysis	Single cloud	Microservice	LSTM	Dependency analysis among microservice units	Improved prediction accuracy	Ignorance of dependency updates	✗	✓	2021	real

Figure A.12: Works on Dependency Analysis of Microservices from [51].



D3.1 : State of the Art in Microservice ADL and Microservice Auto-Scaling Strategies

Finally, Figure A.13 gives the augmented classification of works on *Scaling*.

Ref.	Mechanism	Infrastructure	Application Architecture	Methods	Objectives	Advantages	Limitations	K8S	Scaling	Year	Real/simu/
[65]	Horizontal	Single cloud	Microservice	BO, GP	SLA assurance	High precision and short training time	Poor performance and sub-optimal decisions under workload spikes	✓	✓	2019	real
[93]	Horizontal	Single cloud	Microservice	RF	SLA assurance	Container expansion time reduction	Limited QoS and workload scenarios	✓	✓	2019	real
[70]	Horizontal	Single cloud	Microservice	Heuristic	Cost minimization	resource utilization improvement	High time complexity	✗	✓	2017	simu
[81]	Horizontal	Single cloud	Microservice	Heuristic	Resource efficiency	Avoidance of oscillations under unexpected workload spikes	Rigid scaling mechanisms	✗	✓	2019	simu
[66]	Horizontal	Single cloud	Microservice	Heuristic	SLA assurance	Resource utilization optimization	High time complexity	✓	✓	2020	real
[67]	Horizontal	Single cloud	Microservice	Heuristic	SLA assurance	Lower request loss	High resource usage	✓	✓	2020	real
[74]	Vertical	Single cloud	Microservice	Heuristic	SLO assurance	Improved resource utilization	Simplicity of application datasets	✓	✓	2019	real
[78]	Horizontal	Hybrid cloud	Microservice	SARSA	Resource utilization optimization	SLA assurance	Capability limitation of edge devices	✓	✓	2021	real
[23]	Horizontal	Single cloud	Microservice	Heuristic	QoS assurance	Resource utilization optimization	Implicit evaluation of computational costs and time complexity	✗	✓	2021	real
[24]	Hybrid	Single cloud	Microservice	Actor-critic	Resource utilization optimization	SLO violation mitigation	Limited scalability and anomaly detection	✓	✓	2020	real

Figure A.13: Works on Dependency Analysis of Microservices from [51].



References

- [1] *Microservices Adoption in 2020*. <https://www.oreilly.com/radar/microservices-adoption-in-2020/>.
- [2] *Technology Trends for 2023*. <https://www.oreilly.com/radar/technology-trends-for-2023/>.
- [3] Nicola Dragoni, Saverio Giallorenzo, Alberto Lluch Lafuente, Manuel Mazzara, Fabrizio Montesi, Ruslan Mustafin, Larisa Safina, and Bertrand Meyer. "Microservices: Yesterday, Today, and Tomorrow". In: *Present and Ulterior Software Engineering*. Cham: Springer International Publishing, 2017, pp. 195–216. ISBN: 978-3-319-67425-4. DOI: [10.1007/978-3-319-67425-4_12](https://doi.org/10.1007/978-3-319-67425-4_12). URL: https://doi.org/10.1007/978-3-319-67425-4_12.
- [4] Fowler Martin. *Microservices, a definition of this new architectural term*. <http://web.archive.org/web/20080207010024/http://www.808multimedia.com/winnt/kernel.htm>. Accessed: 2023-09-30. 2014.
- [5] Manuel Mazzara, Nicola Dragoni, Antonio Bucchiarone, Alberto Giarretta, Stephan T. Larsen, and Schahram Dustdar. "Microservices: Migration of a Mission Critical System". In: *IEEE Transactions on Services Computing* 14.5 (2021), pp. 1464–1477. DOI: [10.1109/TSC.2018.2889087](https://doi.org/10.1109/TSC.2018.2889087).
- [6] Thomas Erl. *SOA Principles of Service Design (Paperback)*. 1st. USA: Prentice Hall Press, 2016. ISBN: 0134695518.
- [7] Mark Richards. *Microservices vs. service-oriented architecture*. O'Reilly Media Sebastopol, 2015.
- [8] Jacopo Soldani, Damian Andrew Tamburri, and Willem-Jan Van Den Heuvel. "The pains and gains of microservices: A Systematic grey literature review". In: *Journal of Systems and Software* 146 (2018), pp. 215–232. ISSN: 0164-1212. DOI: <https://doi.org/10.1016/j.jss.2018.09.082>. URL: <https://www.sciencedirect.com/science/article/pii/S0164121218302139>.
- [9] Eric Evans. *Domain-driven design: tackling complexity in the heart of software*. Addison-Wesley Professional, 2004.
- [10] Fowler Martin. *Bounded Context*. <https://martinfowler.com/bliki/BoundedContext.html>. Accessed: 2023-09-30. 2014.
- [11] Chris Richardson. *Microservices patterns: with examples in Java*. Simon and Schuster, 2018.
- [12] Mario Villamizar, Oscar Garcés, Harold Castro, Mauricio Verano, Lorena Salamanca, Rubby Casallas, and Santiago Gil. "Evaluating the monolithic and the microservice architecture pattern to deploy web applications in the cloud". In: *2015 10th Computing Colombian Conference (10CCC)*. 2015, pp. 583–590. DOI: [10.1109/ColumbianCC.2015.7333476](https://doi.org/10.1109/ColumbianCC.2015.7333476).
- [13] Nuno Gonçalves, Diogo Faustino, António Rito Silva, and Manuel Portela. "Monolith Modularization Towards Microservices: Refactoring and Performance Trade-offs". In: *2021 IEEE 18th International Conference on Software Architecture Companion (ICSAC)*. 2021, pp. 1–8. DOI: [10.1109/ICSAC-C.2021.00015](https://doi.org/10.1109/ICSAC-C.2021.00015).
- [14] Sara Hassan and Rami Bahsoon. "Microservices and Their Design Trade-Offs: A Self-Adaptive Roadmap". In: *2016 IEEE International Conference on Services Computing (SCC)*. 2016, pp. 813–818. DOI: [10.1109/SCC.2016.113](https://doi.org/10.1109/SCC.2016.113).
- [15] Steve Vestal. *A cursory overview and comparison of four architecture description languages*. Tech. rep. Cite-seer, 1993.
- [16] Technical Committee : ISO/IEC JTC 1/SC 7 Software and systems engineering. *Software, systems and enterprise Architecture Description*. Standard. International Organization for Standardization, Nov. 2022.
- [17] N. Medvidovic and R.N. Taylor. "A classification and comparison framework for software architecture description languages". In: *IEEE Transactions on Software Engineering* 26.1 (2000), pp. 70–93. DOI: [10.1109/32.825767](https://doi.org/10.1109/32.825767).
- [18] Arie van Deursen, Paul Klint, and Joost Visser. "Domain-Specific Languages: An Annotated Bibliography". In: *SIGPLAN Not.* 35.6 (June 2000), pp. 26–36. ISSN: 0362-1340. DOI: [10.1145/352029.352035](https://doi.org/10.1145/352029.352035). URL: <https://doi.org/10.1145/352029.352035>.



- [19] Paul C. Clements. “A Survey of Architecture Description Languages”. In: *Proceedings of the 8th International Workshop on Software Specification and Design. IWSSD '96*. USA: IEEE Computer Society, 1996, p. 16. ISBN: 0818673613.
- [20] Perri Tarr and Harold Ossher. *Concern Spaces*. <https://web.archive.org/web/20080122043747/http://www.research.ibm.com/hyperspace/ConcernSpaces.htm>. Accessed: 2023-09-30. 2008.
- [21] Peri Tarr, Harold Ossher, William Harrison, and Stanley M. Sutton. “N Degrees of Separation: Multi-Dimensional Separation of Concerns”. In: *Proceedings of the 21st International Conference on Software Engineering. ICSE '99*. Los Angeles, California, USA: Association for Computing Machinery, 1999, pp. 107–119. ISBN: 1581130740. DOI: [10.1145/302405.302457](https://doi.org/10.1145/302405.302457). URL: <https://doi.org/10.1145/302405.302457>.
- [22] José Mara Conejero, Juan Hernández, Elena Jurado, and Klaas van den Berg. “Crosscutting, what is and what is not?: A Formal definition based on a Crosscutting Pattern”. In: *target 1.t2* (2007), t3.
- [23] Jóakim von Kistowski, Simon Eismann, Norbert Schmitt, André Bauer, Johannes Grohmann, and Samuel Kounev. “TeaStore: A Micro-Service Reference Application for Benchmarking, Modeling and Resource Management Research”. In: *Proceedings of the 26th IEEE International Symposium on the Modelling, Analysis, and Simulation of Computer and Telecommunication Systems. MASCOTS '18*. Milwaukee, WI, USA, Sept. 2018.
- [24] Hugo Monfleur and Philippe Merle. “Towards Concern-Oriented Microservice Architecture”. In: *2023 International Conference on Microservices*. 2023.
- [25] Florian Rademacher. “A Language Ecosystem for Modeling Microservice Architecture”. PhD thesis. Kassel, Universität Kassel, Fachbereich Elektrotechnik / Informatik, 2022. DOI: [doi:10.17170/kobra-202209306919](https://doi.org/10.17170/kobra-202209306919).
- [26] Alen Suljkanovi, Branko Milosavljevi, Vladimir Ini, and Igor Dejanovi. “Developing Microservice-Based Applications Using the Silvera Domain-Specific Language”. In: *Applied Sciences* 12.13 (2022), p. 6679.
- [27] Branko Terzic, Vladimir Dimitrieski, Slavica Kordic, Gordana Milosavljevic, and Ivan Lukovic. “Microbuilder: A model-driven tool for the specification of rest microservice architectures”. In: *International Conference on Information Society and Technology*. 2017, pp. 179–184.
- [28] Giona Granchelli, Mario Cardarelli, Paolo Di Francesco, Ivano Malavolta, Ludovico Iovino, and Amleto Di Salle. “Microart: A software architecture recovery tool for maintaining microservice-based systems”. In: *2017 IEEE International Conference on Software Architecture Workshops (ICSAW)*. IEEE. 2017, pp. 298–302.
- [29] *Jolie Programming Language - Official Website*. <https://www.jolie-lang.org/>.
- [30] *JHipster - Fullstack Platform for the Modern Developer!* <https://www.jhipster.tech/>.
- [31] *JHipster Domain Language*. <https://www.jhipster.tech/>.
- [32] *Github - The Compose Specification*. <https://github.com/compose-spec/compose-spec>.
- [33] *Overview | Docker Docs*. <https://docs.docker.com/compose/compose-file/>.
- [34] <https://kubernetes.io/>. <https://kubernetes.io/fr/>.
- [35] Claudio Guidi, Ivan Lanese, Manuel Mazzara, and Fabrizio Montesi. “Microservices: A Language-Based Approach”. In: *Present and Ulterior Software Engineering*. Ed. by Manuel Mazzara and Bertrand Meyer. Cham: Springer International Publishing, 2017, pp. 217–225. ISBN: 978-3-319-67425-4. DOI: [10.1007/978-3-319-67425-4_13](https://doi.org/10.1007/978-3-319-67425-4_13). URL: https://doi.org/10.1007/978-3-319-67425-4_13.
- [36] Gustavo Alonso, Fabio Casati, Harumi Kuno, Vijay Machiraju, Gustavo Alonso, Fabio Casati, Harumi Kuno, and Vijay Machiraju. *Web services*. Springer, 2004.
- [37] *Kompose - Convert your Docker Compose file to Kubernetes or OpenShift*. <https://kompose.io/>. Accessed: 2023-09-30.
- [38] Philip Wizenty, Francisco Ponce, Florian Rademacher, Jacopo Soldani, Hernán Astudillo, Antonio Brogi, and Sabine Sachweh. “Towards resolving security smells in microservices, model-driven”. In: *18th International Conference on Software Technologies (ICSOFT)*. 2023.



- [39] *The C4 model for visualizing software architecture*. <https://c4model.com/>.
- [40] Tomas Cerny, Amr S. Abdelfattah, Vincent Bushong, Abdullah Al Maruf, and Davide Taibi. "Microservice Architecture Reconstruction and Visualization Techniques: A Review". In: *2022 IEEE International Conference on Service-Oriented System Engineering (SOSE)*. 2022, pp. 39–48. DOI: [10.1109/SOSE55356.2022.00011](https://doi.org/10.1109/SOSE55356.2022.00011).
- [41] Mark Badger, Timothy Grance, Robert Patt-Corner, and Jeffrey Voas. *Cloud Computing Synopsis and Recommendations*. en. 2012-05-29 2012. DOI: <https://doi.org/10.6028/NIST.SP.800-146>.
- [42] Y. Al-Dhuraibi, F. Paraiso, N. Djarallah, and P. Merle. "Elasticity in Cloud Computing: State of the Art and Research Challenges". In: *IEEE Transactions on Services Computing* 11.2 (2018), pp. 430–447. URL: <https://hal.inria.fr/hal-01529654>.
- [43] J.O. Kephart and D.M. Chess. "The vision of autonomic computing". In: *Computer* 36.1 (2003), pp. 41–50. DOI: [10.1109/MC.2003.1160055](https://doi.org/10.1109/MC.2003.1160055).
- [44] *An Architectural Blueprint for Autonomic Computing*. Tech. rep. IBM, June 2005.
- [45] Chris Richardson. *Service Instance per VM Pattern*. <https://microservices.io/patterns/deployment/service-per-vm.html>.
- [46] Hokeun Lim, Younghun Kim, and Kyoungjae Sun. "Service Management in Virtual Machine and Container Mixed Environment using Service Mesh". In: *2021 International Conference on Information Networking (ICOIN)*. 2021, pp. 528–530. DOI: [10.1109/ICOIN50884.2021.9333888](https://doi.org/10.1109/ICOIN50884.2021.9333888).
- [47] Chris Richardson. *Service Instance per Container Pattern*. <https://microservices.io/patterns/deployment/service-per-container.html>.
- [48] Vindeep Singh and Sateesh K Peddoju. "Container-based microservice architecture for cloud applications". In: *2017 International Conference on Computing, Communication and Automation (ICCCA)*. 2017, pp. 847–852. DOI: [10.1109/CCAA.2017.8229914](https://doi.org/10.1109/CCAA.2017.8229914).
- [49] João Nunes, Thiago Bianchi, Anderson Iwasaki, and Elisa Nakagawa. "State of the Art on Microservices Autoscaling: An Overview". In: *Anais do XLVIII Seminário Integrado de Software e Hardware*. Evento Online: SBC, 2021, pp. 30–38. DOI: [10.5753/semish.2021.15804](https://doi.org/10.5753/semish.2021.15804). URL: <https://sol.sbc.org.br/index.php/semish/article/view/15804>.
- [50] Shamsuddeen Rabiou, Chan Huah Yong, and Sharifah Mashita Syed Mohamad. "A Cloud-based Container Microservices: A Review on Load-balancing and Auto-scaling Issues". In: *International Journal of Data Science* 3.2 (2022), pp. 80–92. DOI: [10.18517/ijods.3.2.80-92.2022](https://doi.org/10.18517/ijods.3.2.80-92.2022).
- [51] Zhiheng Zhong, Minxian Xu, Maria Alejandra Rodriguez, Chengzhong Xu, and Rajkumar Buyya. "Machine Learning-Based Orchestration of Containers: A Taxonomy and Future Directions". In: *ACM Comput. Surv.* 54.10s (Sept. 2022). ISSN: 0360-0300. DOI: [10.1145/3510415](https://doi.org/10.1145/3510415). URL: <https://doi.org/10.1145/3510415>.
- [52] Siti Nuraishah Agos Jawaddi, Muhammad Hamizan Johari, and Azlan Ismail. "A review of microservices autoscaling with formal verification perspective". In: *Software: Practice and Experience* 52.11 (2022), pp. 2476–2495. DOI: <https://doi.org/10.1002/spe.3135>. eprint: <https://onlinelibrary.wiley.com/doi/pdf/10.1002/spe.3135>. URL: <https://onlinelibrary.wiley.com/doi/abs/10.1002/spe.3135>.
- [53] Mohamed Fourati, Soumaya Marzouk, and Mohamed Jmaiel. *Cloud Elasticity of Microservices-based Applications: A Survey*. Feb. 2024. DOI: [10.21203/rs.3.rs-3925329/v1](https://doi.org/10.21203/rs.3.rs-3925329/v1).
- [54] Satish Narayana Srirama, Mainak Adhikari, and Souvik Paul. "Application deployment using containers with auto-scaling for microservices in cloud environment". In: *Journal of Network and Computer Applications* 160 (2020), p. 102629. ISSN: 1084-8045. DOI: <https://doi.org/10.1016/j.jnca.2020.102629>. URL: <https://www.sciencedirect.com/science/article/pii/S108480452030103X>.
- [55] Alireza Goli, Nima Mahmoudi, Hamzeh Khazaei, and Omid Ardakanian. "A Holistic Machine Learning-Based Autoscaling Approach for Microservice Applications". In: Feb. 2021. DOI: [10.5220/0010407701900198](https://doi.org/10.5220/0010407701900198).
- [56] Guangba Yu and al. "Microscaler: Automatic Scaling for Microservices with an Online Learning Approach". In: *ICWS. IEEE*, 2019, pp. 68–75. DOI: [10.1109/ICWS.2019.00023](https://doi.org/10.1109/ICWS.2019.00023).



- [57] Jingze Lv, Mingchang Wei, and Yang Yu. "A Container Scheduling Strategy Based on Machine Learning in Microservice Architecture". In: *2019 IEEE International Conference on Services Computing (SCC)*. 2019, pp. 65–71. DOI: [10.1109/SCC.2019.00023](https://doi.org/10.1109/SCC.2019.00023).
- [58] Peng Kang and Palden Lama. "Robust Resource Scaling of Containerized Microservices with Probabilistic Machine learning". In: *2020 IEEE/ACM 13th International Conference on Utility and Cloud Computing (UCC)*. 2020, pp. 122–131. DOI: [10.1109/UCC48980.2020.00031](https://doi.org/10.1109/UCC48980.2020.00031).
- [59] Laszlo Toka, Gergely Dobreff, Balazs Fodor, and Balazs Sonkoly. "Adaptive AI-based auto-scaling for Kubernetes". In: *2020 20th IEEE/ACM International Symposium on Cluster, Cloud and Internet Computing (CCGRID)*. 2020, pp. 599–608. DOI: [10.1109/CCGrid49817.2020.00-33](https://doi.org/10.1109/CCGrid49817.2020.00-33).
- [60] Vladimir Podolskiy, Michael Mayo, Abigail Koay, Michael Gerndt, and Panos Patros. "Maintaining SLOs of Cloud-Native Applications Via Self-Adaptive Resource Sharing". In: *2019 IEEE 13th International Conference on Self-Adaptive and Self-Organizing Systems (SASO)*. 2019, pp. 72–81. DOI: [10.1109/SASO.2019.00018](https://doi.org/10.1109/SASO.2019.00018).
- [61] Ming Yan, XiaoMeng Liang, ZhiHui Lu, Jie Wu, and Wei Zhang. "HANSEL: Adaptive horizontal scaling of microservices using Bi-LSTM". In: *Applied Soft Computing* 105 (2021), p. 107216. ISSN: 1568-4946. DOI: <https://doi.org/10.1016/j.asoc.2021.107216>. URL: <https://www.sciencedirect.com/science/article/pii/S1568494621001393>.
- [62] Yanqi Zhang, Weizhe Hua, Zhuangzhuang Zhou, G. Edward Suh, and Christina Delimitrou. "Sinan: ML-Based and QoS-Aware Resource Management for Cloud Microservices". In: *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*. ASPLOS '21. Virtual, USA: Association for Computing Machinery, 2021, pp. 167–181. ISBN: 9781450383172. DOI: [10.1145/3445814.3446693](https://doi.org/10.1145/3445814.3446693). URL: <https://doi-org.ins2i.bib.cnrs.fr/10.1145/3445814.3446693>.
- [63] Haoran Qiu, Subho S. Banerjee, Saurabh Jha, Zbigniew T. Kalbarczyk, and Ravishankar K. Iyer. "FIRM: An Intelligent Fine-grained Resource Management Framework for SLO-Oriented Microservices". In: *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. USENIX Association, Nov. 2020, pp. 805–825. ISBN: 978-1-939133-19-9. URL: <https://www.usenix.org/conference/osdi20/presentation/qiu>.
- [64] Zhe Yang, Phuong Nguyen, Haiming Jin, and Klara Nahrstedt. "MIRAS: Model-based Reinforcement Learning for Microservice Resource Allocation over Scientific Workflows". In: *2019 IEEE 39th International Conference on Distributed Computing Systems (ICDCS)*. 2019, pp. 122–132. DOI: [10.1109/ICDCS.2019.00021](https://doi.org/10.1109/ICDCS.2019.00021).
- [65] Ziliang Wang, Shiyi Zhu, Jianguo Li, Wei Jiang, K. K. Ramakrishnan, Meng Yan, Xiaohong Zhang, and Alex X. Liu. "DeepScaling: Autoscaling Microservices With Stable CPU Utilization for Large Scale Production Cloud Systems". In: *IEEE/ACM Transactions on Networking* (2024), pp. 1–16. DOI: [10.1109/TNET.2024.3400953](https://doi.org/10.1109/TNET.2024.3400953).
- [66] Imad Alawe, Yassine Hadjadj-Aoul, Adlen Ksentini, Philippe Bertin, César Viho, and Davy Darche. "Smart scaling of the 5G core network: an RNN-based approach". In: *Globecom 2018 - IEEE Global Communications Conference*. Abu Dhabi, United Arab Emirates: IEEE, Dec. 2018, pp. 1–6. URL: <https://inria.hal.science/hal-01934057>.
- [67] Imad Alawe, Yassine Hadjadj-Aoul, Adlen Ksentini, Philippe Bertin, and Davy Darche. "On the scalability of 5G core network: The AMF case". In: *2018 15th IEEE Annual Consumer Communications and Networking Conference (CCNC)*. 2018, pp. 1–6.
- [68] Imad Alawe, Yassine Hadjadj-Aoul, Adlen Ksentini, Philippe Bertin, Cesar Viho, and Davy Darche. "Smart Scaling of the 5G Core Network: An RNN-Based Approach". In: *2018 IEEE Global Communications Conference (GLOBECOM)*. 2018, pp. 1–6. DOI: [10.1109/GLOCOM.2018.8647590](https://doi.org/10.1109/GLOCOM.2018.8647590).
- [69] Imad Alawe, Adlen Ksentini, Yassine Hadjadj-Aoul, and Philippe Bertin. "Improving Traffic Forecasting for 5G Core Network Scalability: A Machine Learning Approach". In: *IEEE Network* 32.6 (2018), pp. 42–49. DOI: [10.1109/MNET.2018.1800104](https://doi.org/10.1109/MNET.2018.1800104).
- [70] Yahuza Bello, Alaa Awad Abdellatif, Mhd Saria Allahham, Ahmed Refaey Hussein, Aiman Erbad, Amr Mohamed, and Mohsen Guizani. "B5G: Predictive Container Auto-Scaling for Cellular Evolved Packet Core". In: *IEEE Access* 9 (2021), pp. 158204–158214. DOI: [10.1109/ACCESS.2021.3126048](https://doi.org/10.1109/ACCESS.2021.3126048).



- [71] Tejas Subramanya and Roberto Riggio. “Machine Learning-Driven Scaling and Placement of Virtual Network Functions at the Network Edges”. In: *2019 IEEE Conference on Network Softwarization (NetSoft)*. 2019, pp. 414–422. DOI: [10.1109/NETSOFT.2019.8806631](https://doi.org/10.1109/NETSOFT.2019.8806631).